

TP2 — Infrastructure as Code

avec **Terraform** pour l'application **ShopEasy**

Automatiser le déploiement d'une architecture Azure reproductible, versionnée et maîtrisée par le coût.

Binôme

Louis SCARFONE

Maxence BOURRAGUÉ

Cadre

Mastère Dev Manager Full Stack — RNCP niveau 7

Cas fil rouge : **ShopEasy**

Azure for Students — région swedencentral

14

ateliers + quiz

21

ressources Terraform

≈ 46,5 \$

coût mensuel estimé

Sommaire

Table des matières

1	Atelier 1 — Initialisation du projet Terraform	2
1.1	Vérification de l'environnement	2
1.2	Arborescence du projet	3
1.2.1	Rôle de chaque fichier	4
1.3	Fichier .gitignore	4
1.4	Point de contrôle — pourquoi ne jamais publier terraform.tfstate	5
1.5	Protection des valeurs sensibles	5
1.6	Capture	6
2	Atelier 2 — Déclaration des providers	7
2.1	versions.tf	7
2.2	providers.tf	8
2.3	Initialisation — terraform init	8
2.4	Formatage — terraform fmt	8
2.5	Validation — terraform validate	8
3	Atelier 3 — Paramétrage du projet	9
3.1	Variables d'entrée — variables.tf	9
3.2	Valeurs de l'environnement — terraform.tfvars	10
3.3	Locals et tags — locals.tf	11
3.4	Validation	11
4	Atelier 4 — Groupe de ressources et réseau	12
4.1	Authentification du provider	12
4.2	Ressources réseau — network.tf	12
4.3	Prévisualisation — terraform plan	13
4.4	Application — terraform apply	14
4.5	Vérification (Azure CLI et état Terraform)	14
4.6	Captures portail	15
4.7	Questions	16
5	Atelier 5 — Sécurisation du réseau avec un NSG	17
5.1	NSG, règles et association — security.tf	17
5.2	Prévisualisation et application	18
5.3	Vérification (Azure CLI)	18
5.4	Captures portail	19
5.5	Analyse de sécurité des flux	20
5.6	Question de recul — l'accès SSH en production	20
6	Atelier 6 — Déploiement des deux machines virtuelles Linux	21

6.1	Script cloud-init — templates/cloud-init.yml	21
6.2	IP publiques, interfaces et VM — compute.tf	21
6.3	Plan et application	23
6.4	Vérification (Azure CLI et accès web)	23
6.5	Captures portail et navigateur	24
6.6	Questions	25
7	Atelier 7 — Mise en place du Load Balancer	26
7.1	IP publique, LB, pool, sonde et règle — loadbalancer.tf	26
7.2	Plan et application	27
7.3	Vérification et test de répartition	27
7.4	Captures	28
7.5	Analyse	29
8	Atelier 8 — Ajout d'un Storage Account	30
8.1	Suffixe aléatoire, Storage Account et container — storage.tf	30
8.2	Plan, application et vérification	31
8.3	Captures portail	32
8.4	Questions FinOps et sécurité	33
8.5	Note de sécurité — accès public au niveau du compte	33
9	Atelier 9 — Outputs et validation	34
9.1	Sorties exposées — outputs.tf	34
9.2	Application et lecture des sorties	35
9.3	Checklist technique	36
9.4	Page web via le Load Balancer	37
10	Atelier 10 — Modification de l'infrastructure	38
10.1	Ajout du tag cost_center dans locals.tf	38
10.2	Prévisualisation — terraform plan	38
10.3	Application et idempotence	39
10.4	Analyse du plan	39
11	Atelier 11 — Observation d'une dérive Terraform	40
11.1	Création d'une dérive volontaire	40
11.2	Détection — terraform plan	40
11.3	Réconciliation — terraform apply	40
11.4	Capture portail	41
11.5	Analyse	41
12	Atelier 12 — Préparation d'un state distant	42
12.1	Le state local et ses limites	42
12.2	Structure cible — backend Azure Storage	43
12.3	Questions	43
13	Atelier 13 — Analyse coût, sécurité et maintenabilité	44
13.1	Analyse FinOps	44
13.2	Analyse de sécurité	45

13.3	Maintenabilité	45
14	Atelier 14 — Nettoyage de l'environnement	46
14.1	Pourquoi détruire l'environnement?	46
14.2	Prévisualisation — terraform plan -destroy	47
14.3	Exécution et vérification	47
15	Note technique — synthèse des choix	48
15.1	Contexte	48
15.2	Architecture déployée	49
15.3	Choix techniques	49
15.4	Sécurité	50
15.5	FinOps	50
15.6	Gouvernance et cycle de vie	50
15.7	Amélioration proposée (mise en autonomie — Option B : Azure Bastion)	50
16	Quiz de validation	51

1 Atelier 1 — Initialisation du projet Terraform

🎯 **Objectif** — poser les fondations d'un projet Terraform propre, lisible et sécurisé pour ShopEasy.

📦 **Livrable attendu** — l'arborescence du projet et le fichier `.gitignore`, avec la justification de la non-publication du `terraform.tfstate`.

Le TP1 a conçu et déployé manuellement (Azure CLI) l'architecture cible de ShopEasy. Le TP2 reprend cette architecture mais la décrit en **Infrastructure as Code** avec **Terraform** : l'infrastructure devient un ensemble de fichiers `.tf` versionnés, relus, appliqués puis détruits de façon reproductible. Cet atelier met en place le squelette du projet; aucune ressource Azure n'est encore créée.

1.1 Vérification de l'environnement

L'environnement local est vérifié avant toute initialisation :

>_ bash

```
terraform version
git --version
az version --query '"azure-cli"' -o tsv
az account show --query '{name:name, state:state, user:user.name}' -o json
```

>_ console

```
Terraform v1.15.7
on darwin_arm64

git version 2.50.1 (Apple Git-155)

2.87.0

{ "name": "Azure for Students", "state": "Enabled", "user": "louis.scarfone@efrei.net" }
```

Prérequis	Attendu	Constaté	État
Terraform CLI	≥ 1.6	v1.15.7	✓
Azure CLI	présent + connecté	2.87.0, session active	✓
Git	présent	2.50.1	✓
Abonnement Azure	formation, actif	Azure for Students, Enabled	✓
Clé SSH publique	~/.ssh/id_rsa.pub	présente (générée au TP1)	✓

Terraform n'était pas installé au départ : ajouté via `brew install hashicorp/tap/terraform` — le tap officiel HashiCorp, requis depuis le passage de Terraform sous licence **BSL**.

1.2 Arborescence du projet

>_ bash

```
mkdir -p tp2-terraform-azure/templates
cd tp2-terraform-azure
touch versions.tf providers.tf variables.tf locals.tf \
    network.tf security.tf compute.tf loadbalancer.tf \
    storage.tf outputs.tf terraform.tfvars templates/cloud-init.yml
```

>_ text

```
tp2-terraform-azure/
|-- .gitignore           exclusions Git (state, secrets, cache)
|-- versions.tf          versions Terraform + providers requis
|-- providers.tf         configuration du provider azurerm
|-- variables.tf         variables d'entree (parametrage du projet)
|-- locals.tf            valeurs calculees (prefixe de nommage, tags communs)
|-- network.tf           Resource Group, VNet, subnet
|-- security.tf          Network Security Group + regles + association
|-- compute.tf           IP publiques, interfaces reseau, 2 VM Linux (cloud-init)
|-- loadbalancer.tf      IP publique LB, Load Balancer, backend pool, sonde, regle
|-- storage.tf           suffixe aleatoire, Storage Account, container Blob
|-- outputs.tf           sorties exposees apres deploiement
|-- terraform.tfvars     valeurs concretes (IGNORE par git -- contient l'IP admin)
`-- templates/
    |-- cloud-init.yml   script d'installation Nginx au premier demarrage des VM
```

Le projet est découpé par **responsabilité** (réseau / sécurité / calcul / stockage / sorties) plutôt qu'en un seul fichier `main.tf`. Ce découpage améliore la lisibilité et la maintenabilité : chaque fichier regroupe un type de ressources cohérent et peut être relu indépendamment. Les fichiers `.tf` sont créés vides à ce stade et seront complétés atelier par atelier.

Rôle de chaque fichier

Fichier	Rôle
versions.tf	Version minimale de Terraform + providers requis (azurerm, random) avec contraintes de version.
providers.tf	Configuration du provider azurerm (bloc features {}).
variables.tf	Paramètres configurables : projet, environnement, région, admin, clé SSH, CIDR SSH.
locals.tf	Valeurs internes réutilisables : préfixe de nommage, tags communs.
network.tf	Resource Group + Virtual Network + subnet applicatif.
security.tf	NSG, règles entrantes (HTTP, SSH restreint) et association au subnet.
compute.tf	IP publiques, interfaces réseau et 2 VM Linux provisionnées par cloud-init.
loadbalancer.tf	IP publique + Load Balancer L4 (frontend, backend pool, sonde, règle).
storage.tf	Suffixe aléatoire + Storage Account privé + container Blob versionné.
outputs.tf	Sorties utiles (nom du RG, IP du LB, IP des VM, nom du Storage).
terraform.tfvars	Valeurs concrètes de l'environnement dev (exclu du dépôt).
templates/cloud-init.yml	Configuration cloud-init (installation et démarrage Nginx, page de test).

1.3 Fichier .gitignore

<pre> >_ gitignore # Repertoire de travail Terraform (providers telecharges, cache, plugins) .terraform/ # Fichiers de STATE : peuvent contenir des donnees sensibles en clair *.tfstate *.tfstate.* # Variables converties en JSON (peuvent embarquer des valeurs sensibles) *.tfvars.json # Valeurs concretes d'environnement : terraform.tfvars contient le CIDR SSH # = IP publique reelle de l'admin. Seul l'exemple anonymise est versionne. terraform.tfvars # Fichier de verrouillage des providers (choix du TP : non versionne) </pre>
--

```
.terraform.lock.hcl

# Logs de crash Terraform
crash.log
crash.*.log
```

Motif ignoré	Raison
.terraform/	Cache local volumineux (binaires des providers), recréé par terraform init.
*.tfstate	Le state peut contenir des secrets en clair et l’inventaire complet de l’infrastructure (cf. §4).
*.tfvars.json	Variables au format JSON pouvant contenir des valeurs sensibles.
terraform.tfvars	Contient l’IP publique réelle de l’administrateur — seul l’exemple anonymisé est versionné (§5).
.terraform.lock.hcl	Fichier de verrouillage des providers — non versionné selon le choix du TP (§5).
crash.*.log	Journaux de crash temporaires, sans valeur de suivi.

1.4 Point de contrôle — pourquoi ne jamais publier terraform.tfstate

Le fichier terraform.tfstate est une représentation JSON de toutes les ressources gérées par Terraform et de leurs attributs. Le publier dans un dépôt non sécurisé est dangereux pour quatre raisons :

1. **Présence possible de secrets en clair.** Terraform stocke les attributs renvoyés par Azure, y compris des valeurs sensibles : clés d’accès Storage, mots de passe admin, chaînes de connexion, tokens SAS. Même si aucun secret n’est dans les .tf, il peut se retrouver dans le state.
2. **Exposition de l’inventaire complet** de l’infrastructure (noms, identifiants, IP, plages de subnets, règles réseau) — une cartographie de reconnaissance pour un attaquant.
3. **Rôle de source de vérité.** S’il est exposé, corrompu ou modifié, Terraform peut détruire ou recréer des ressources par erreur, ou perdre la trace de ce qu’il gère.
4. **Incompatibilité avec le travail d’équipe en local** : conflits de fusion et absence de verrouillage → corruption du fichier.

La solution professionnelle consiste à externaliser le state dans un **backend distant** (Storage Account Azure dédié) avec chiffrement, RBAC et verrouillage — objet de l’Atelier 12. En attendant, le .gitignore garantit que le state ne quitte jamais le poste local.

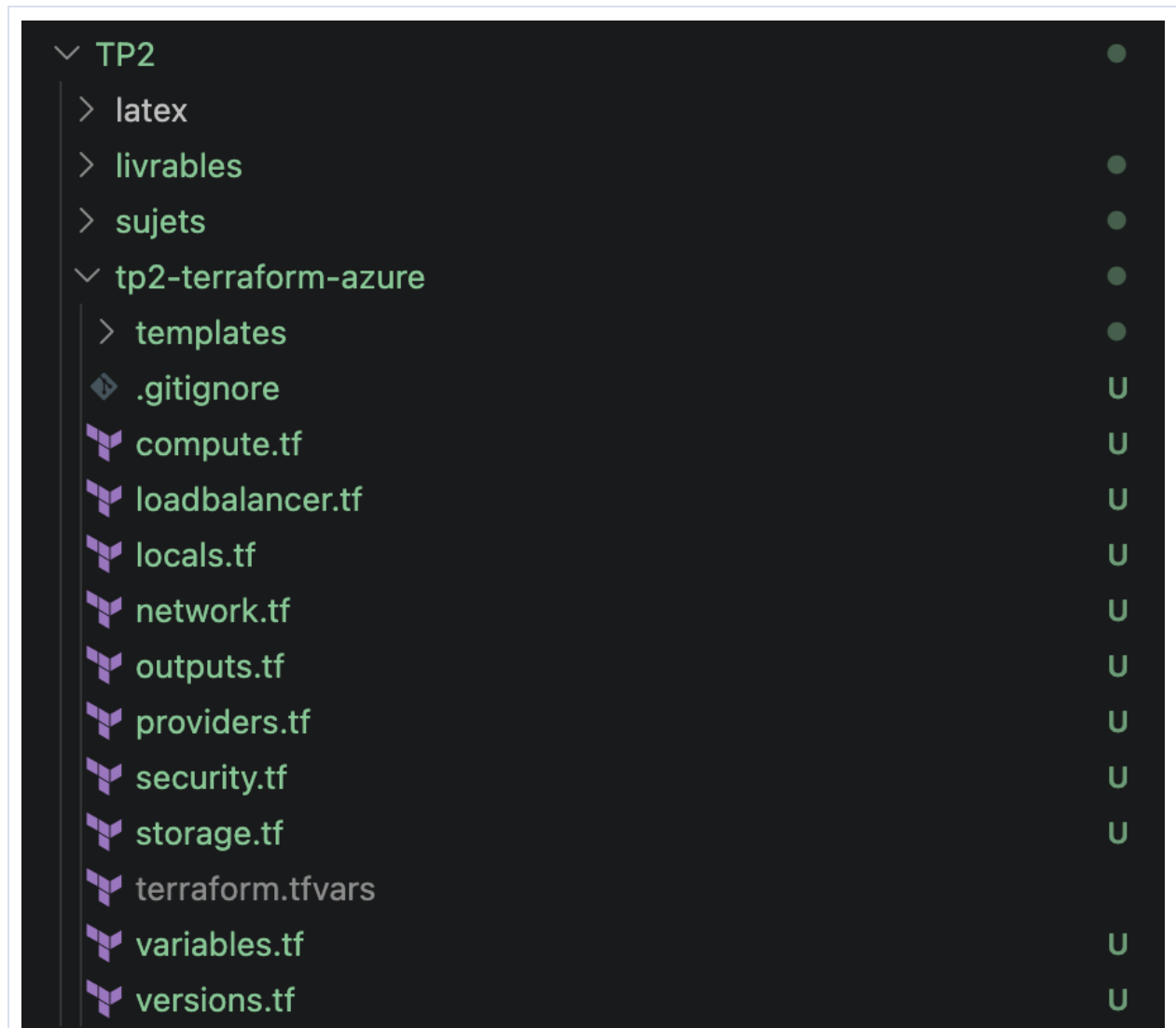
1.5 Protection des valeurs sensibles

Exclusion de terraform.tfvars. Le .gitignore fourni par le TP n’exclut pas terraform.tfvars. Or ce fichier contiendra allowed_ssh_cidr = l’IP publique réelle de l’administrateur. Conformément à la règle du cours (« ne jamais mettre de valeur sensible dans un fichier versionné ») et à la pratique

du TP1 (masquage de l'IP), `terraform.tfvars` est exclu du dépôt; un `terraform.tfvars.example` anonymisé est versionné à sa place (Atelier 3).

Fichier `.terraform.lock.hcl`. Le TP demande de l'ignorer. En projet d'équipe réel, HashiCorp recommande plutôt de le versionner pour figer les versions de providers et garantir des déploiements identiques. Le choix du TP est suivi ici, la bonne pratique étant notée.

1.6 Capture



Arborescence du projet `tp2-terraform-azure` dans l'éditeur : 10 fichiers `.tf`, `templates/cloud-init.yml` et `.gitignore`.

✓ État après l'atelier

- Arborescence `tp2-terraform-azure/` créée : 10 fichiers `.tf` (vides), `terraform.tfvars`, `templates/cloud-init.yml` et `.gitignore`.
- Environnement validé : Terraform v1.15.7, Azure CLI 2.87.0 (*Azure for Students*), Git 2.50.1.
- `.gitignore` en place : `state`, `secrets` et `terraform.tfvars` (IP admin) exclus du dépôt; point

de contrôle tfstate traité.

- Région `francecentral` et taille `B1s` du TP non disponibles sur Azure for Students → adaptées en `swedencentral` et `B2ats_v2` (Ateliers 3 et 6).
- Prêt pour l'Atelier 2 — déclaration des providers.

2 Atelier 2 — Déclaration des providers

🎯 **Objectif** — déclarer la version de Terraform et les providers requis, puis initialiser le projet.

📦 **Livrable attendu** — `versions.tf`, `providers.tf` et le résultat de `terraform validate`.

2.1 `versions.tf`

```
>_ hcl

terraform {
  required_version = ">= 1.6.0"

  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 4.0"
    }
    random = {
      source = "hashicorp/random"
      version = "~> 3.6"
    }
  }
}
```

Élément	Signification
<code>required_version = ">= 1.6.0"</code>	Le projet exige Terraform 1.6 ou plus (version installée : 1.15.7).
<code>azurerm</code>	Provider standard pour Azure : RG, VNet, NSG, VM, Load Balancer, Storage.
<code>random</code>	Génère un suffixe aléatoire pour rendre le nom du Storage Account globalement unique (Atelier 8).
<code>version = "~> 4.0"</code>	Opérateur pessimiste : autorise <code>>= 4.0.0</code> et <code>< 5.0.0</code> (toutes les 4.x).
<code>version = "~> 3.6"</code>	Autorise <code>>= 3.6.0</code> et <code>< 4.0.0</code> .

L'opérateur `~>` fige la branche majeure d'un provider : les correctifs 4.x sont récupérés automatiquement, mais une version 5.0 (susceptible d'introduire des *breaking changes*) est exclue.

2.2 providers.tf

```
>_ hcl

provider "azurerm" {
  features {}
}
```

Le bloc `features {}` est obligatoire pour le provider `azurerm`, même vide : il active le comportement par défaut du provider. L'authentification est héritée de la session Azure CLI (`az login`) déjà active, ce qui évite d'inscrire un identifiant dans le code.

💡 Authentification azurerm v4.x

Le provider `azurerm v4.x` requiert un `subscription_id` lors des opérations `plan` et `apply` (pas pour `init` ni `validate`). Il sera fourni via la variable d'environnement `ARM_SUBSCRIPTION_ID` à l'Atelier 4, pour conserver `providers.tf` minimal et sans identifiant en dur.

2.3 Initialisation — terraform init

```
>_ console

Initializing provider plugins found in the configuration...
- Finding hashicorp/random versions matching "~> 3.6"...
- Finding hashicorp/azurerm versions matching "~> 4.0"...
- Installing hashicorp/random v3.9.0...
- Installed hashicorp/random v3.9.0 (signed by HashiCorp)
- Installing hashicorp/azurerm v4.78.0...
- Installed hashicorp/azurerm v4.78.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above.

Terraform has been successfully initialized!
```

`terraform init` a résolu et téléchargé les providers (vérifiés par signature HashiCorp) : `azurerm v4.78.0` et `random v3.9.0`. La commande a créé le dossier de travail `.terraform/` et le fichier `.terraform.lock.hcl` qui fige les versions exactes retenues.

2.4 Formatage — terraform fmt

La commande ne renvoie aucune sortie : tous les fichiers `.tf` sont déjà conformes au style canonique HCL. `terraform fmt` n'affiche que les fichiers qu'il modifie ; une sortie vide est le résultat attendu.

2.5 Validation — terraform validate

```
>_ console

Success! The configuration is valid.
```

La configuration est syntaxiquement et structurellement valide. terraform valide et travaille en local : il vérifie la cohérence du code sans contacter Azure ni l'existence réelle des ressources.

✓ État après l'atelier

- versions.tf : Terraform >= 1.6.0, providers azurerm ~> 4.0 et random ~> 3.6.
- providers.tf : provider azurerm configuré (features {}), authentification héritée d'az login.
- terraform init réussi : azurerm v4.78.0 et random v3.9.0 installés, lock file créé; validate: Success.
- Prêt pour l'Atelier 3 — paramétrage du projet.

3 Atelier 3 — Paramétrage du projet

🎯 **Objectif** — rendre le projet réutilisable en externalisant les valeurs configurables et en centralisant le nommage et les tags.

📦 **Livrable attendu** — variables.tf, terraform.tfvars et locals.tf (variables, valeurs d'environnement, locals et tags).

3.1 Variables d'entrée — variables.tf

```
>_ hcl

variable "project" {
  description = "Nom court du projet"
  type        = string
  default     = "shopeasy"
}

variable "environment" {
  description = "Nom de l'environnement"
  type        = string
  default     = "dev"
}

variable "location" {
  description = "Region Azure cible"
  type        = string
  default     = "francecentral"
}

variable "admin_username" {
  description = "Utilisateur administrateur Linux"
  type        = string
  default     = "azureuser"
}

variable "ssh_public_key_path" {
  description = "Chemin local vers la cle publique SSH"
```

```

    type      = string
  }

  variable "allowed_ssh_cidr" {
    description = "CIDR autorise pour l'accès SSH"
    type        = string
  }

```

Variable	Type	Défaut	Rôle
project	string	shopeasy	Nom court du projet ; alimente le préfixe et les tags.
environment	string	dev	Environnement ; alimente le préfixe et les tags.
location	string	francecentral	Région Azure cible (surchargée par terraform.tfvars, §2).
admin_username	string	azureuser	Compte administrateur Linux des VM.
ssh_public_key_path	string	(requis)	Chemin de la clé publique SSH injectée dans les VM.
allowed_ssh_cidr	string	(requis)	Plage CIDR autorisée en SSH (IP admin en /32).

Les variables `ssh_public_key_path` et `allowed_ssh_cidr` sont **sans valeur par défaut** : elles sont propres à l'environnement et doivent être fournies via `terraform.tfvars`. Externaliser ces paramètres évite tout codage en dur et permet de réutiliser le même code pour d'autres environnements.

3.2 Valeurs de l'environnement — terraform.tfvars

⚠ Adaptation de la région (Azure for Students)

Le TP fixe `location = "francecentral"`. Cette région est **bloquée** par la policy *Allowed resource deployment regions* de l'abonnement *Azure for Students* (contrainte déjà rencontrée au TP1). La région déployable retenue est **swedencentral**. La valeur est définie dans `terraform.tfvars` — chargé automatiquement par Terraform — sans modifier le défaut de `variables.tf`.

Valeurs retenues (modèle anonymisé `terraform.tfvars.example`) :

```

>_ hcl

project      = "shopeasy"
environment  = "dev"
location     = "swedencentral" # francecentral bloquee sur Azure for Students
admin_username = "azureuser"
ssh_public_key_path = "~/ssh/id_rsa.pub"
allowed_ssh_cidr  = "X.X.X.X/32" # IP publique de l'administrateur (masquee)

```

Le fichier réel `terraform.tfvars` renseigne `allowed_ssh_cidr` avec l'**IP publique réelle** de l'administrateur, en `/32`. Conformément à la règle «aucune valeur sensible dans un fichier versionné», `terraform.tfvars` est **exclu du dépôt**; seul `terraform.tfvars.example`, anonymisé, est versionné.

3.3 Locals et tags — `locals.tf`

```
>_ hcl

locals {
  # Prefixe de nommage commun a toutes les ressources : "shopeasy-dev"
  prefix = "${var.project}-${var.environment}"

  # Tags de gouvernance appliques a chaque ressource
  common_tags = {
    project      = var.project
    environment  = var.environment
    owner        = "formation"
    managed_by   = "terraform"
  }
}
```

prefix vaut `shopeasy-dev` et est réutilisé pour nommer chaque ressource de façon cohérente (`rg-shopeasy-dev`, `vnet-shopeasy-dev`, `nsg-shopeasy-dev-web...`). `common_tags` regroupe les tags de gouvernance appliqués à toutes les ressources :

Tag	Valeur	Usage
project	shopeasy	Rattachement applicatif.
environment	dev	Distinction dev / test / prod.
owner	formation	Équipe responsable.
managed_by	terraform	Indique que la ressource est gérée en IaC (toute modification manuelle est une dérive).

Centraliser le nommage et les tags dans `locals.tf` applique le principe *DRY* : une seule source de vérité, modifiable en un point, propagée à l'ensemble des ressources.

3.4 Validation

```
>_ console

$ terraform fmt      # aucune sortie : fichiers deja conformes
$ terraform validate
Success! The configuration is valid.

# Controle de la protection de l'IP par .gitignore
terraform.tfvars      -> IGNORE par git (OK, contient l'IP)
terraform.tfvars.example -> VERSIONNE (OK, anonymise)
```

La configuration est valide et le fichier contenant l'IP réelle est bien exclu du dépôt.

✓ État après l'atelier

- `variables.tf` : 6 variables d'entrée (projet, environnement, région, admin, clé SSH, CIDR SSH).
- `terraform.tfvars` : environnement dev, **région swedencentral** (adaptation Azure for Students), IP admin en /32 — fichier **exclu du dépôt**.
- `terraform.tfvars.example` : modèle **anonymisé** versionné; `locals.tf` : préfixe shopeasy-dev + 4 tags de gouvernance.
- **Prêt pour l'Atelier 4 — Resource Group et réseau (premier apply).**

4 Atelier 4 — Groupe de ressources et réseau

🎯 **Objectif** — créer le groupe de ressources et le réseau privé de ShopEasy avec Terraform, et réaliser le premier apply.

📦 **Livrable attendu** — `network.tf` (RG + VNet + subnet) et la preuve de création (plan, apply, portail).

🌐 **Région** — swedencentral • RG rg-shopeasy-dev, VNet 10.20.0.0/16.

4.1 Authentification du provider

L'authentification vers Azure est héritée de la session Azure CLI (`az login`). Le provider `azurerm` v4.x exige en plus l'identifiant d'abonnement, fourni par variable d'environnement, **sans l'inscrire dans le code** :

```
>_ bash
```

```
export ARM_SUBSCRIPTION_ID=$(az account show --query id -o tsv)
```

4.2 Ressources réseau — `network.tf`

```
>_ hcl
```

```
resource "azurerm_resource_group" "main" {
  name      = "rg-${local.prefix}"
  location  = var.location
  tags      = local.common_tags
}

resource "azurerm_virtual_network" "main" {
  name            = "vnet-${local.prefix}"
  address_space   = ["10.20.0.0/16"]
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags            = local.common_tags
}
```

```
resource "azurerm_subnet" "web" {
  name                = "snet-web"
  resource_group_name = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes    = ["10.20.1.0/24"]
}
```

Le VNet et le subnet **réfèrent** les attributs du Resource Group : Terraform en déduit le **graphe de dépendances** (RG d'abord, puis VNet, puis subnet). Le nommage s'appuie sur `local.prefix` et chaque élément porte `local.common_tags`.

Ressource	Nom	Valeur clé
azurerm_resource_group.main	rg-shopeasy-dev	région swedencentral
azurerm_virtual_network.main	vnet-shopeasy-dev	espace 10.20.0.0/16
azurerm_subnet.web	snet-web	préfixe 10.20.1.0/24

4.3 Prévisualisation — terraform plan

>_ console

Terraform will perform the following actions:

```
# azurerm_resource_group.main will be created
+ resource "azurerm_resource_group" "main" {
  + id          = (known after apply)
  + location    = "swedencentral"
  + name       = "rg-shopeasy-dev"
  + tags       = {
    + "environment" = "dev"
    + "managed_by"  = "terraform"
    + "owner"       = "formation"
    + "project"     = "shopeasy"
  }
}

# azurerm_subnet.web will be created
+ resource "azurerm_subnet" "web" {
  + address_prefixes = [ "10.20.1.0/24" ]
  + name            = "snet-web"
  + resource_group_name = "rg-shopeasy-dev"
  + virtual_network_name = "vnet-shopeasy-dev"
}

# azurerm_virtual_network.main will be created
+ resource "azurerm_virtual_network" "main" {
  + address_space = [ "10.20.0.0/16" ]
  + location      = "swedencentral"
  + name         = "vnet-shopeasy-dev"
  + resource_group_name = "rg-shopeasy-dev"
  + tags         = { ... 4 tags ... }
}
```



```
Plan: 3 to add, 0 to change, 0 to destroy.
```

Le plan annonce **3 ressources à créer**, aucune modification ni destruction — résultat attendu pour un premier déploiement.

4.4 Application — terraform apply

>_ console

```
azurerm_resource_group.main: Creating...
azurerm_resource_group.main: Creation complete after 26s
↪ [id=/subscriptions/e67b15f9-.../resourceGroups/rg-shopeasy-dev]
azurerm_virtual_network.main: Creating...
azurerm_virtual_network.main: Creation complete after 6s [id=.../virtualNetworks/vnet-shopeasy-dev]
azurerm_subnet.web: Creating...
azurerm_subnet.web: Creation complete after 5s [id=.../subnets/snet-web]

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
```

L'identifiant d'abonnement est masqué (e67b15f9-...) dans les ID de ressources.

4.5 Vérification (Azure CLI et état Terraform)

>_ console

```
$ az group show -n rg-shopeasy-dev --query '{name,location,provisioningState:...,tags}'
{ "location": "swedencentral", "name": "rg-shopeasy-dev",
  "provisioningState": "Succeeded",
  "tags": { "environment":"dev","managed_by":"terraform","owner":"formation","project":"shopeasy" } }

$ az network vnet show -g rg-shopeasy-dev -n vnet-shopeasy-dev --query '{...}'
{ "addressSpace": [ "10.20.0.0/16" ], "location": "swedencentral",
  "name": "vnet-shopeasy-dev", "provisioningState": "Succeeded" }
-> subnet snet-web : addressPrefix 10.20.1.0/24, state Succeeded

$ terraform state list
azurerm_resource_group.main
azurerm_subnet.web
azurerm_virtual_network.main
```

Les trois ressources sont en provisioningState: Succeeded, avec la bonne région, le bon plan d'adressage et les quatre tags de gouvernance. L'état Terraform recense exactement les trois ressources gérées.

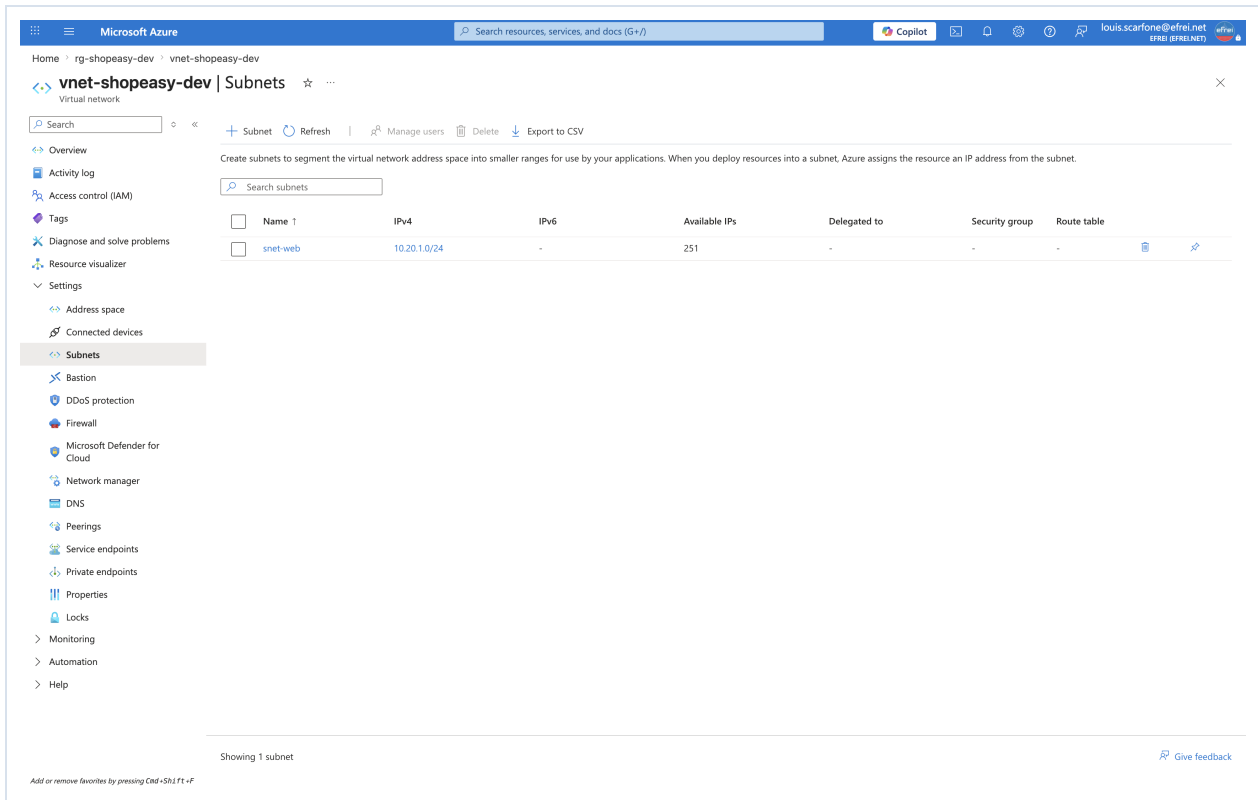
4.6 Captures portail

The screenshot shows the Microsoft Azure portal interface. The top navigation bar includes the Microsoft Azure logo, a search bar, and the user profile 'louis.scarfone@efrei.net'. The main content area displays the 'Overview' page for the Resource Group 'rg-shopeasy-dev'. The left sidebar contains a navigation menu with options like Overview, Activity log, Access control (IAM), Tags, Resource visualizer, Events, Settings, Cost Management, Monitoring, Automation, and Help. The main content area shows the 'Essentials' section with details about the Subscription (Azure for Students), Subscription ID, Deployments (No deployments), Location (Sweden Central), and Tags (environment: dev, managed_by: terraform). Below this, the 'Resources' section is visible, showing a table with columns for Name, Type, and Location. The table lists one resource: 'vnet-shopeasy-dev' of type 'Virtual network' located in 'Sweden Central'. The bottom of the page shows 'Showing 1 - 1 of 1. Display count: auto' and a 'Give feedback' link.

Vue d'ensemble du Resource Group rg-shopeasy-dev (région Sweden Central) — subscription ID floutée.

The screenshot shows the Microsoft Azure portal interface, specifically the 'Tags' page for the Resource Group 'rg-shopeasy-dev'. The top navigation bar is the same as the previous screenshot. The main content area displays the 'Tags' page, which includes a search bar, a 'Delete all' button, and a 'Feedback' link. Below this, there is a section titled 'Tags are name/value pairs that enable you to categorize resources and view consolidated billing by applying the same tag to multiple resources and resource groups. Tag names are case insensitive, but tag values are case sensitive. Learn more about tags?'. A warning message states: 'Do not enter names or values that could make your resources less secure or that contain personal/sensitive information because tag data will be replicated globally.' Below this, there is a table with columns for 'Name' and 'Value'. The table lists four tags: 'environment' with value 'dev', 'managed_by' with value 'terraform', 'owner' with value 'formation', and 'project' with value 'shopeasy'. At the bottom of the page, there are buttons for 'Apply' and 'Discard changes'.

Les 4 tags de gouvernance appliqués au Resource Group.



Le subnet *snet-web* (10.20.1.0/24) dans le VNet *vnet-shopeasy-dev*.

4.7 Questions

1. Pourquoi le VNet utilise-t-il une plage privée ?

La plage 10.20.0.0/16 appartient aux **adresses privées RFC 1918**, non routables sur Internet. Un VNet est un **réseau privé** : une plage privée évite tout conflit avec des adresses publiques, garde les communications internes **isolées**, et permet une **segmentation maîtrisée** par subnets. L'exposition vers l'extérieur se fait **explicitement** via des IP publiques et un Load Balancer (Ateliers 6 et 7), jamais par le VNet lui-même.

2. Que faudrait-il ajouter pour isoler une base de données dans un subnet dédié ?

Un **second subnet** (ex. *snet-data*, 10.20.2.0/24) réservé à la couche données, associé à son **propre NSG** n'autorisant l'accès à la base (port 1433/5432) **que depuis le subnet web** et refusant le reste. Idéalement, la base est jointe via un **Private Endpoint** (aucune exposition publique). C'est l'application de la **segmentation réseau** et du **moindre privilège**.

3. Pourquoi séparer les fichiers Terraform au lieu de tout placer dans `main.tf` ?

Terraform charge **tous** les fichiers `.tf` d'un répertoire quel que soit leur nom : le découpage est purement **organisationnel**, sans coût fonctionnel. Regrouper les ressources par **responsabilité** rend le projet **lisible**, **navigable** et **maintenable**, et réduit les conflits de fusion en équipe.

✓ État après l'atelier

- `network.tf` créé; terraform apply : **3 ressources ajoutées** (rg-shopeasy-dev, vnet-shopeasy-dev, snet-web), toutes en Succeeded.
- Région `swedencentral`, plan d'adressage 10.20.0.0/16 (subnet web 10.20.1.0/24), 4 tags

de gouvernance.

- Prêt pour l'Atelier 5 — sécurisation du réseau avec un NSG.

5 Atelier 5 — Sécurisation du réseau avec un NSG

🎯 **Objectif** — filtrer les flux réseau du subnet web en n'autorisant que les ouvertures nécessaires.

📦 **Livrable attendu** — `security.tf` (NSG + règles + association) et l'analyse de sécurité des flux.

5.1 NSG, règles et association — `security.tf`

>_ hcl

```
resource "azurerm_network_security_group" "web" {
  name            = "nsg-${local.prefix}-web"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags           = local.common_tags

  security_rule {
    name            = "Allow-HTTP"
    priority        = 100
    direction      = "Inbound"
    access         = "Allow"
    protocol       = "Tcp"
    source_port_range = "*"
    destination_port_range = "80"
    source_address_prefix = "Internet"
    destination_address_prefix = "*"
  }

  security_rule {
    name            = "Allow-SSH-Admin"
    priority        = 110
    direction      = "Inbound"
    access         = "Allow"
    protocol       = "Tcp"
    source_port_range = "*"
    destination_port_range = "22"
    source_address_prefix = var.allowed_ssh_cidr
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "web" {
  subnet_id            = azurerm_subnet.web.id
  network_security_group_id = azurerm_network_security_group.web.id
}
```

Règle	Prio	Sens	Accès	Port	Source	Rôle
Allow-HTTP	100	In	Allow	80	Internet	Accès HTTP à l'application web de test
Allow-SSH-Admin	110	In	Allow	22	<IP_ADMIN>/32	Administration Linux limitée à l'IP admin

La source SSH n'est jamais codée en dur : elle provient de la variable `allowed_ssh_cidr` (définie dans `terraform.tfvars`, exclu du dépôt). L'association rattache le NSG au subnet `snet-web` : toutes les VM qui y seront déployées héritent du filtrage.

5.2 Prévisualisation et application

Le plan annonce **2 ressources à créer** (NSG + association). Extrait (IP admin masquée) :

```
>_ console

# azurerem_network_security_group.web will be created
+ security_rule = [
  + { name = "Allow-SSH-Admin", priority = 110, direction = "Inbound",
    access = "Allow", protocol = "Tcp", destination_port_range = "22",
    source_address_prefix = "<IP_ADMIN>/32" },
  + { name = "Allow-HTTP", priority = 100, direction = "Inbound",
    access = "Allow", protocol = "Tcp", destination_port_range = "80",
    source_address_prefix = "Internet" },
]

Plan: 2 to add, 0 to change, 0 to destroy.
azurerem_network_security_group.web: Creation complete after 2s [id=../nsg-shopeasy-dev-web]
azurerem_subnet_network_security_group_association.web: Creation complete after 6s
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

5.3 Vérification (Azure CLI)

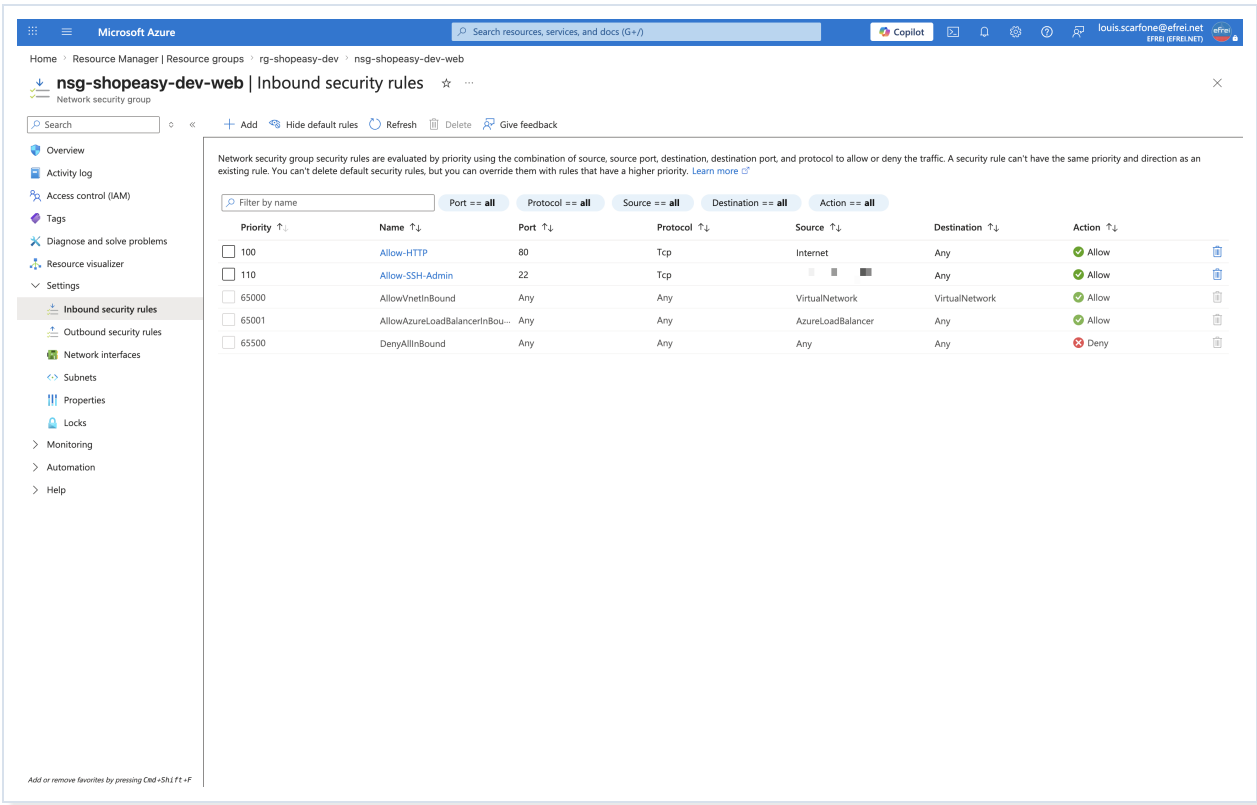
```
>_ console

$ az network nsg rule list -g rg-shopeasy-dev --nsg-name nsg-shopeasy-dev-web ...
Name          Priority Direction Access Proto Port Source
Allow-HTTP    100      Inbound  Allow  Tcp   80   Internet
Allow-SSH-Admin 110      Inbound  Allow  Tcp   22   <IP_ADMIN>/32

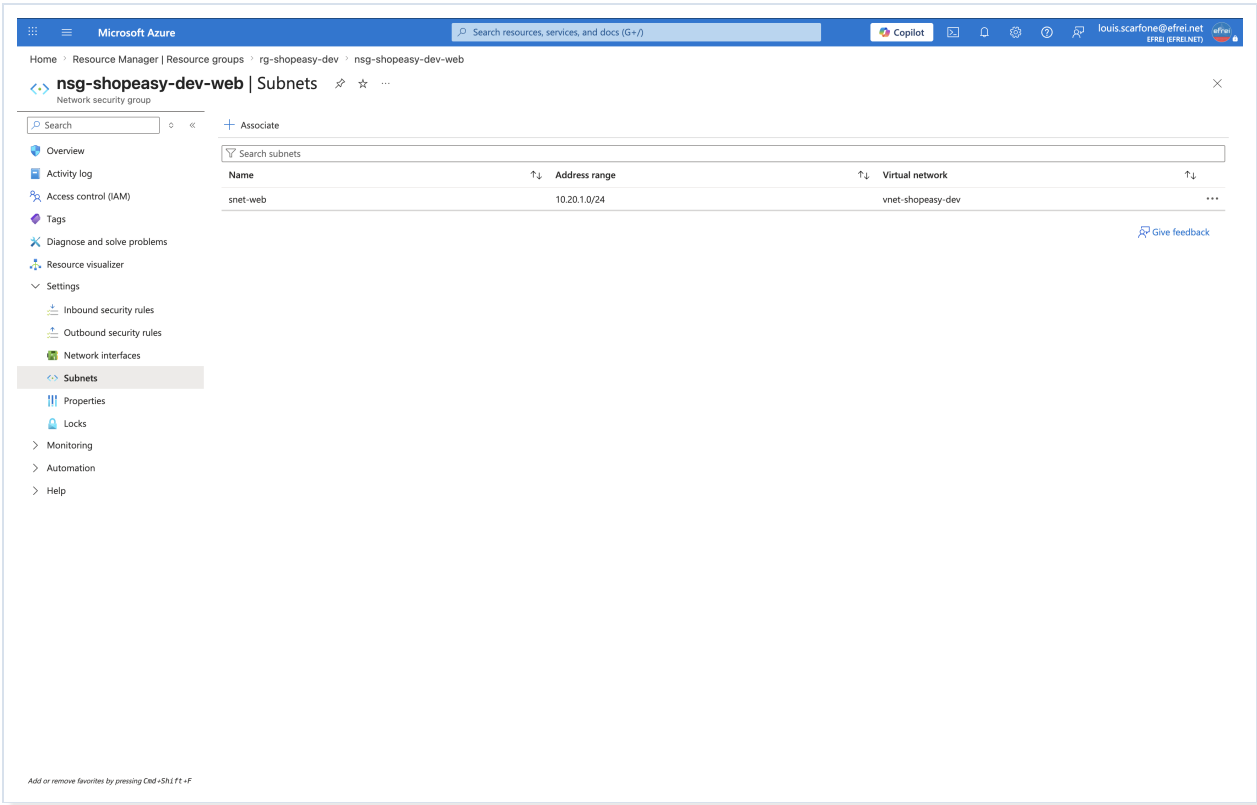
$ az network vnet subnet show ... -n snet-web --query '{subnet,nsg_associe}'
{ "nsg_associe": ".../networkSecurityGroups/nsg-shopeasy-dev-web", "subnet": "snet-web" }
```

Les deux règles sont actives et le subnet `snet-web` est associé au NSG `nsg-shopeasy-dev-web`. La source de la règle SSH est restreinte à l'IP de l'administrateur (masquée).

5.4 Captures portail



Règles entrantes du NSG nsg-shopeasy-dev-web (Allow-HTTP et Allow-SSH-Admin) — source SSH (IP admin) floutée.



Association du NSG nsg-shopeasy-dev-web au subnet snet-web.

5.5 Analyse de sécurité des flux

Flux	Autorisé ?	Justification	Risque résiduel
Internet → HTTP (80)	Oui	Règle Allow-HTTP; expose l'application web de test.	Trafic en clair . En prod : HTTPS (443) + Application Gateway + WAF.
SSH (22) depuis l'IP admin	Oui	Règle Allow-SSH-Admin limitée à l'IP admin en /32.	Dépend d'une IP fixe. En prod : Azure Bastion / VPN, JIT, MFA.
SSH (22) depuis tout Internet	Non	Aucune règle ne l'autorise; DenyAllInBound (65500) bloque le reste.	Quasi nul tant que le CIDR admin reste étroit (/32).
Tout trafic sortant	Oui	Azure autorise l'Outbound par défaut (AllowVnetOutBound, AllowInternetOutBound).	Exfiltration possible. En prod : restreindre l'Outbound aux destinations légitimes.

Un NSG applique des règles par défaut invisibles : AllowVnetInBound, AllowAzureLoadBalancerInBound, DenyAllInBound en entrée. Les règles personnalisées (prio < 65000) sont évaluées **avant** ces défauts.

5.6 Question de recul — l'accès SSH en production

Exposer le port SSH (22) directement sur Internet, même restreint à une IP, reste à **éviter en production**. Les approches recommandées :

- **Azure Bastion** : connexion SSH/RDP via le portail, **sans IP publique** sur les VM;
- **VPN site-to-site** ou point-to-site : administration par le réseau privé;
- **Jump server / bastion host** contrôlé et audité;
- **Accès just-in-time** (Defender for Cloud) : ouverture à la demande, pour une durée et une IP limitées.

L'objectif commun est de **supprimer l'exposition permanente** du SSH au profit d'un accès privé, temporaire et tracé.

✓ État après l'atelier

- security.tf créé : NSG nsg-shopeasy-dev-web (2 règles entrantes) + association au subnet snet-web; apply : **2 ressources ajoutées**.
- HTTP (80) ouvert depuis Internet; SSH (22) **restreint à l'IP admin**; reste de l'entrant bloqué par défaut.
- **Prêt pour l'Atelier 6 — déploiement des deux machines virtuelles Linux.**

6 Atelier 6 — Déploiement des deux machines virtuelles Linux

🎯 **Objectif** — déployer deux serveurs web Linux identiques, provisionnés automatiquement par cloud-init.

📦 **Livrable attendu** — templates/cloud-init.yml, compute.tf (IP publiques, interfaces réseau, VM) et la preuve d'accès web.

⚠️ Adaptation imposée par Azure for Students (taille de VM)

Le TP fixe size = "Standard_B1s", **indisponible** sur l'abonnement *Azure for Students* (contrainte déjà résolue au TP1). La taille retenue est **Standard_B2ats_v2** (2 vCPU, série burstable Basv2), **officiellement recommandée par Microsoft** comme alternative à B1s. Le quota Basv2 (10 vCPU) couvre les $2 \times 2 = 4$ vCPU nécessaires.

6.1 Script cloud-init — templates/cloud-init.yml

```
>_ yaml

#cloud-config
package_update: true
packages:
  - nginx
write_files:
  - path: /var/www/html/index.html
    content: |
      <html>
        <body>
          <h1>ShopEasy - serveur web ${server_index}</h1>
          <p>Déploiement realise avec Terraform sur Azure.</p>
        </body>
      </html>
runcmd:
  - systemctl enable nginx
  - systemctl restart nginx
```

Ce script est exécuté **au premier démarrage** de chaque VM : il installe **Nginx**, écrit une page index.html puis active et démarre le service. Le marqueur `${server_index}` est injecté par Terraform (§2) pour **personnaliser la page par serveur**.

6.2 IP publiques, interfaces et VM — compute.tf

```
>_ hcl

resource "azurerm_public_ip" "web" {
  count          = 2
  name           = "pip-${local.prefix}-web-${count.index + 1}"
  location       = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method = "Static"
  sku            = "Standard"
  tags           = local.common_tags
}
```



```

}

resource "azurerm_network_interface" "web" {
  count                = 2
  name                 = "nic-${local.prefix}-web-${count.index + 1}"
  location             = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                 = local.common_tags

  ip_configuration {
    name                       = "internal"
    subnet_id                 = azurerm_subnet.web.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id       = azurerm_public_ip.web[count.index].id
  }
}

resource "azurerm_linux_virtual_machine" "web" {
  count                = 2
  name                 = "vm-${local.prefix}-web-${count.index + 1}"
  location             = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  size                 = "Standard_B2ats_v2" # B1s indisponible (Azure for Students)
  admin_username       = var.admin_username
  network_interface_ids = [ azurerm_network_interface.web[count.index].id ]

  admin_ssh_key {
    username   = var.admin_username
    public_key = file(pathexpand(var.ssh_public_key_path))
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  custom_data = base64encode(templatefile("${path.module}/templates/cloud-init.yml", {
    server_index = count.index + 1
  }))

  tags = local.common_tags
}

```

- **count = 2** déploie deux exemplaires de chaque ressource, indexés par `count.index` (0 et 1). Chaque VM est reliée à **sa** NIC et **son** IP via `[count.index]`.
- **templatefile(...)** lit le `cloud-init.yml` et y injecte `server_index = count.index + 1`, puis `base64encode` encode le résultat pour `custom_data`.
- **pathexpand(...)** : la fonction `file()` ne développe pas `~`; `pathexpand` convertit `~/ssh/id_rsa.pub` en chemin absolu.

6.3 Plan et application

>_ console

Plan: 6 to add, 0 to change, 0 to destroy. (2 IP publiques, 2 NIC, 2 VM)

azurerm_public_ip.web[0..1]: Creation complete after ~3s

azurerm_network_interface.web[0..1]: Creation complete after ~5s

azurerm_linux_virtual_machine.web[1]: Creation complete after 49s [id=.../vm-shopeasy-dev-web-2]

azurerm_linux_virtual_machine.web[0]: Creation complete after 50s [id=.../vm-shopeasy-dev-web-1]

Apply complete! Resources: 6 added, 0 changed, 0 destroyed.

6.4 Vérification (Azure CLI et accès web)

Nom	Taille	IP publique	IP privée	État
vm-shopeasy-dev-web-1	Standard_B2ats_v2	135.225.73.252	10.20.1.5	VM running
vm-shopeasy-dev-web-2	Standard_B2ats_v2	135.225.74.4	10.20.1.4	VM running

Test HTTP de chaque page (depuis le poste admin) :

>_ console

\$ curl http://135.225.73.252

<html><body><h1>ShopEasy - serveur web 1</h1>

<p>Déploiement realise avec Terraform sur Azure.</p></body></html>

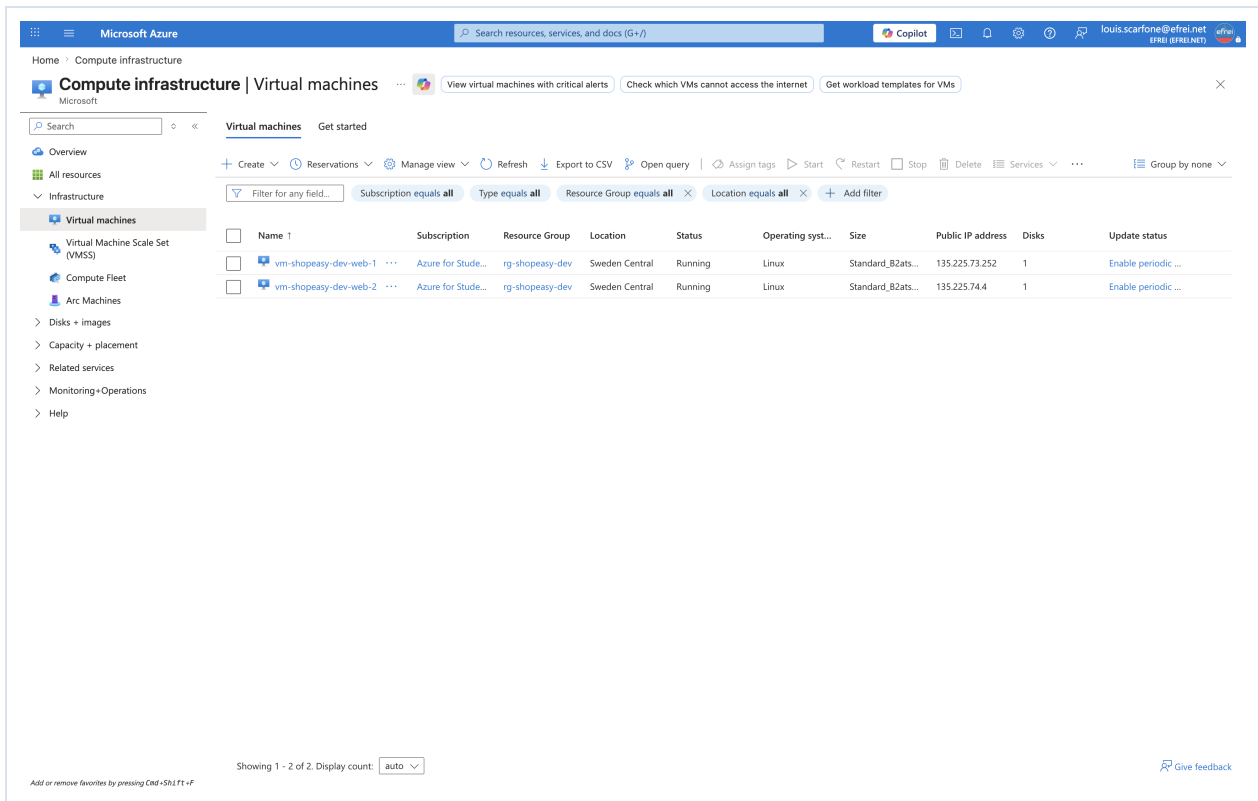
\$ curl http://135.225.74.4

<html><body><h1>ShopEasy - serveur web 2</h1>

<p>Déploiement realise avec Terraform sur Azure.</p></body></html>

Les deux VM sont VM running, en Standard_B2ats_v2, et chaque page identifie son serveur (1 et 2) — ce qui permettra de vérifier la répartition de charge à l'Atelier 7. L'état Terraform recense les **11 ressources** gérées.

6.5 Captures portail et navigateur



Les deux VM `vm-shopeasy-dev-web-1/2` (VM running, `Standard_B2ats_v2`).



Page web servie par `vm-web-1` (`http://135.225.73.252`) : « serveur web 1 ».



Page web servie par vm-web-2 (<http://135.225.74.4>) : « serveur web 2 ».

6.6 Questions

1. À quoi sert `count` dans cette configuration ?

`count = 2` crée **deux exemplaires** de la ressource, indexés par `count.index` (0 et 1). Cela permet de déployer plusieurs serveurs **sans dupliquer le code** (DRY). L'index sert à **nommer** les ressources (`web-1`, `web-2`) et à **relier** chaque VM à la bonne interface et à la bonne IP.

2. Quel est le rôle de `custom_data` ?

`custom_data` transmet à la VM un script **cloud-init** (encodé en base64) exécuté **au premier démarrage**. Ici, il installe Nginx et publie la page de test, sans aucune intervention manuelle ni connexion SSH. Couplé à `templatefile`, il **personnalise** la page de chaque serveur.

3. Pourquoi une VM `Standard_B2ats_v2` (à défaut de `B1s`) est-elle acceptable en formation ?

Une VM **burstable de petite taille** (série B) suffit à un environnement de **test/formation** : charge faible, pas de SLA de production, **coût minimal** ($\approx 0,01$ \$/h). Sur *Azure for Students*, `B1s` est indisponible : `B2ats_v2` est la taille **recommandée** par Microsoft en remplacement.

4. Pourquoi ne pas conserver des IP publiques directes sur les VM en production ?

Une IP publique par VM **expose chaque machine à Internet** (surface d'attaque, scans, brute-force), **coûte en continu** et complique la gouvernance. En production, les VM restent en **IP privée** : l'entrée se fait par un **Load Balancer / Application Gateway** et l'administration par **Azure Bastion**. Ici, les IP publiques sont conservées pour **tester chaque VM individuellement**.

✓ État après l'atelier

- `templates/cloud-init.yml` + `compute.tf` créés : **2 IP publiques, 2 interfaces réseau, 2 VM Linux Standard_B2ats_v2**; `apply` : **6 ressources ajoutées**.
- Nginx installé automatiquement par cloud-init; pages distinctes (« serveur web 1 » / « serveur web 2 ») accessibles en HTTP.
- État Terraform synchronisé (**11 ressources gérées**).
- **Prêt pour l'Atelier 7 — mise en place du Load Balancer.**

7 Atelier 7 — Mise en place du Load Balancer

🎯 **Objectif** — répartir le trafic web entrant sur les deux VM via un Load Balancer public.

📦 **Livrable attendu** — loadbalancer.tf et la preuve de répartition (alternance des réponses entre les 2 serveurs).

🌐 **Région** — swedencentral • Load Balancer lb-shopeasy-dev-web (SKU Standard, public).

7.1 IP publique, LB, pool, sonde et règle — loadbalancer.tf

```
>_ hcl

resource "azurerm_public_ip" "lb" {
  name            = "pip-${local.prefix}-lb"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method = "Static"
  sku             = "Standard"
  tags            = local.common_tags
}

resource "azurerm_lb" "web" {
  name            = "lb-${local.prefix}-web"
  location        = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  sku             = "Standard"
  tags            = local.common_tags
  frontend_ip_configuration {
    name            = "public-frontend"
    public_ip_address_id = azurerm_public_ip.lb.id
  }
}

resource "azurerm_lb_backend_address_pool" "web" {
  name            = "backend-web"
  loadbalancer_id = azurerm_lb.web.id
}

resource "azurerm_network_interface_backend_address_pool_association" "web" {
  count            = 2
  network_interface_id = azurerm_network_interface.web[count.index].id
  ip_configuration_name = "internal"
  backend_address_pool_id = azurerm_lb_backend_address_pool.web.id
}

resource "azurerm_lb_probe" "http" {
  name            = "http-probe"
  loadbalancer_id = azurerm_lb.web.id
  protocol        = "Http"
  request_path    = "/"
  port            = 80
}

resource "azurerm_lb_rule" "http" {
```

```

name                = "http-rule"
loadbalancer_id     = azurerm_lb.web.id
protocol            = "Tcp"
frontend_port       = 80
backend_port        = 80
frontend_ip_configuration_name = "public-frontend"
backend_address_pool_ids = [azurerm_lb_backend_address_pool.web.id]
probe_id            = azurerm_lb_probe.http.id
}

```

Ressource	Rôle
azurerm_public_ip.lb	IP publique statique du frontend du LB.
azurerm_lb.web	Load Balancer Standard (L4), frontend public-frontend.
azurerm_lb_backend_address_pool.web	Pool des VM cibles (backend-web).
azurerm_network_interface_backend_address_pool_association.web (x2)	Rattache chaque NIC (web-1, web-2) au pool.
azurerm_lb_probe.http	Sonde HTTP sur / port 80 (santé des VM).
azurerm_lb_rule.http	Règle 80 → 80 (Tcp) liant frontend + pool + sonde.

7.2 Plan et application

>_ console

```

Plan: 7 to add, 0 to change, 0 to destroy.
(LB + frontend, backend pool, 2 associations NIC, sonde, regle, IP publique)

azurerm_public_ip.lb: Creation complete after 2s [id=.../pip-shopeasy-dev-lb]
azurerm_lb.web: Creation complete after 11s [id=.../lb-shopeasy-dev-web]
azurerm_lb_probe.http: Creation complete after 11s
azurerm_lb_backend_address_pool.web: Creation complete after 14s
azurerm_network_interface_backend_address_pool_association.web[0..1]: complete after 2s
azurerm_lb_rule.http: Creation complete after 9s

Apply complete! Resources: 7 added, 0 changed, 0 destroyed.

```

7.3 Vérification et test de répartition

IP publique du Load Balancer : 9.223.33.133 (SKU Standard, frontend public-frontend).

Règle	Proto	Front	Back	Sonde	Proto	Port	Chemin
http-rule	Tcp	80	80	http-probe	Http	80	/

Les deux interfaces nic-shopeasy-dev-web-1/2 sont membres du pool backend-web. Test de répartition

— 12 requêtes sur l'IP du Load Balancer :

> console

```
$ for i in $(seq 1 12); do curl -s http://9.223.33.133 | grep -o "serveur web [12]"; done
req 1->web 2 req 2->web 1 req 3->web 1 req 4->web 1 req 5->web 2 req 6->web 1
req 7->web 2 req 8->web 2 req 9->web 2 req 10->web 1 req 11->web 1 req 12->web 2
=> 6 reponses "serveur web 1" / 6 reponses "serveur web 2"
```

💡 Répartition de charge effective

Sur 12 requêtes via le LB, **6 sur web-1 et 6 sur web-2** : le Load Balancer répartit bien le trafic entre les deux VM, et la sonde HTTP confirme leur état de santé. C'est la suppression du point de défaillance unique applicatif via un **point d'entrée unique**.

7.4 Captures

The screenshot shows the Azure portal interface for the resource group 'rg-shopeasy-dev'. The selected resource is 'lb-shopeasy-dev-web', a Load balancer. The left sidebar shows the navigation menu with 'Overview' selected. The main content area displays the 'Essentials' tab, which provides a summary of the load balancer's configuration:

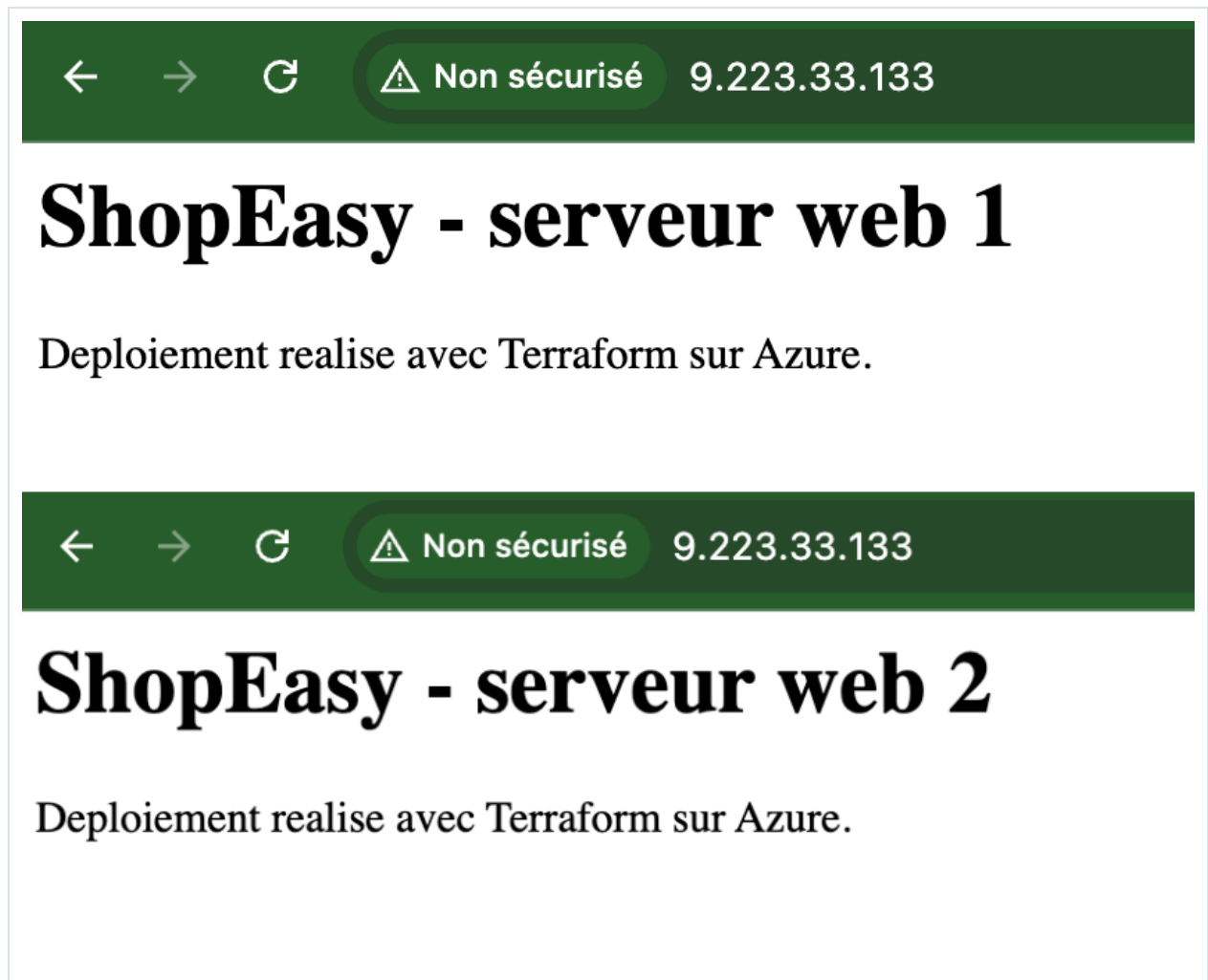
- Resource group:** rg-shopeasy-dev
- Location:** Sweden Central
- Subscription:** Azure for Students
- SKU:** Standard
- Tier:** Regional
- Tags:** environment: dev, managed_by: terraform
- Backend pool:** backend-web (2 virtual machines)
- Load balancing rule:** http-rule (Tcp/80)
- Health probe:** http-probe (Http/80)
- Inbound NAT rules:** -
- Outbound rules:** -
- Frontend IP address:** -

Below the configuration summary, there is a section titled 'Configure high availability and scalability for your applications' with three cards:

- Balance IPv4 and IPv6 addresses:** Native dual-stack endpoints help meet regulatory requirements and address the fast-growing number of devices in mobile and IoT. [Learn more](#)
- Build highly reliable applications:** Load Balancer improves application uptime by routing traffic to healthy nodes. [Learn more](#)
- Secure your networks:** Control network traffic and protect private networks using built-in network address translation (NAT). [Learn more](#)

At the bottom of the page, there is a small text: 'Add or remove favorites by pressing Cmd+Shift+F'.

Vue d'ensemble du Load Balancer lb-shopeasy-dev-web (frontend, backend pool, règle) — subscription ID flouté.



Page web servie via le Load Balancer (<http://9.223.33.133>) ; en rafraîchissant, la réponse alterne entre « serveur web 1 » et « serveur web 2 ».

7.5 Analyse

1. Quel problème résout le Load Balancer ?

Il **répartit le trafic entrant** sur plusieurs VM derrière un **point d'entrée public unique**, évitant la surcharge d'un seul serveur, améliorant la **disponibilité** et permettant de **monter en charge**. Il découple l'adresse publique des serveurs individuels.

2. Que se passe-t-il si une VM devient indisponible ?

La **sonde de santé** (http-probe sur /) cesse de recevoir une réponse 200 et **retire la VM de la rotation**. Le trafic est routé **uniquement vers la (les) VM saine(s)** : le service reste disponible. Dès que la VM repasse la sonde, elle est **réintégrée automatiquement**.

3. Quelle différence avec Azure Application Gateway ?

Azure Load Balancer opère en **couche 4** (TCP/UDP) : répartition par hash, sans comprendre le contenu HTTP. **Application Gateway** opère en **couche 7** (HTTP/HTTPS) : routage par chemin/hôte, **terminaison TLS**, affinité de session et surtout **WAF**. On choisit le LB pour une répartition L4 générique, et l'Application Gateway pour les fonctions L7 (cf. analyse de sécurité).

✓ État après l'atelier

- loadbalancer.tf créé : IP publique + Load Balancer Standard + backend pool (2 VM) + sonde HTTP + règle 80; apply : **7 ressources ajoutées**.
- Test validé : **alternance 6/6** des réponses entre « serveur web 1 » et « serveur web 2 » via l'IP du LB.
- État Terraform : **18 ressources gérées**.
- **Prêt pour l'Atelier 8 — ajout d'un Storage Account.**

8 Atelier 8 — Ajout d'un Storage Account

🎯 **Objectif** — ajouter un stockage documentaire privé et versionné, avec un nom globalement unique.

📦 **Livrable attendu** — storage.tf (suffixe aléatoire + Storage Account + container) et vérifications (privé, versioning, tags).

8.1 Suffixe aléatoire, Storage Account et container — storage.tf

```
>_ hcl

resource "random_string" "suffix" {
  length  = 6
  upper   = false
  special = false
}

resource "azurerm_storage_account" "docs" {
  name                        = "${replace(local.prefix, "-", "")}docs${random_string.suffix.result}"
  resource_group_name        = azurerm_resource_group.main.name
  location                   = azurerm_resource_group.main.location
  account_tier                = "Standard"
  account_replication_type    = "LRS"
  min_tls_version             = "TLS1_2"
  tags                       = local.common_tags
  blob_properties {
    versioning_enabled = true
  }
}

resource "azurerm_storage_container" "documents" {
  name                = "documents"
  storage_account_id = azurerm_storage_account.docs.id
  container_access_type = "private"
}
```

Élément / choix	Raison
<code>random_string.suffix (6 car.)</code>	Le nom d'un Storage Account doit être globalement unique (3-24 car., minuscules/chiffres).
<code>replace(local.prefix, "-", "")</code>	Un Storage Account n'accepte ni tiret ni majuscule → <code>shopeasydevdocs1ac6q9</code> .
<code>account_replication_type = "LRS"</code>	Redondance locale, suffisante en dev (pas de GRS facturée inutilement).
<code>min_tls_version = "TLS1_2"</code>	Refuse les protocoles obsolètes (sécurité en transit).
<code>versioning_enabled = true</code>	Conserve l'historique des objets (récupération après écrasement/suppression).
<code>container_access_type = "private"</code>	Aucun accès anonyme : seuls les appels authentifiés accèdent aux documents.

Le provider `random` génère le suffixe **au moment de l'apply** : sa valeur est « known after apply » dans le plan.

8.2 Plan, application et vérification

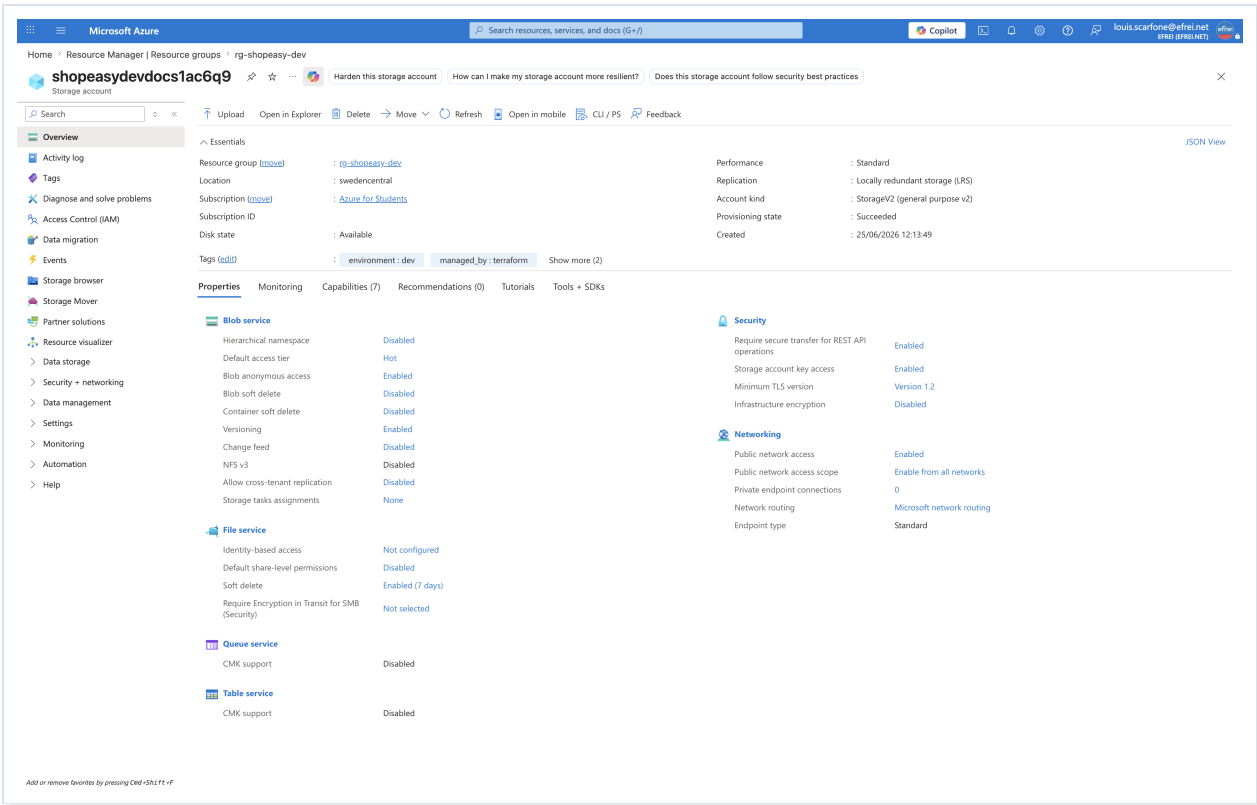
```
>_ console

Plan: 3 to add, 0 to change, 0 to destroy. (random_string + Storage Account + container)
random_string.suffix: Creation complete [id=1ac6q9]
azurerm_storage_account.docs: Creation complete after 1m6s [id=.../shopeasydevdocs1ac6q9]
azurerm_storage_container.documents: Creation complete after 11s [id=.../containers/documents]
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

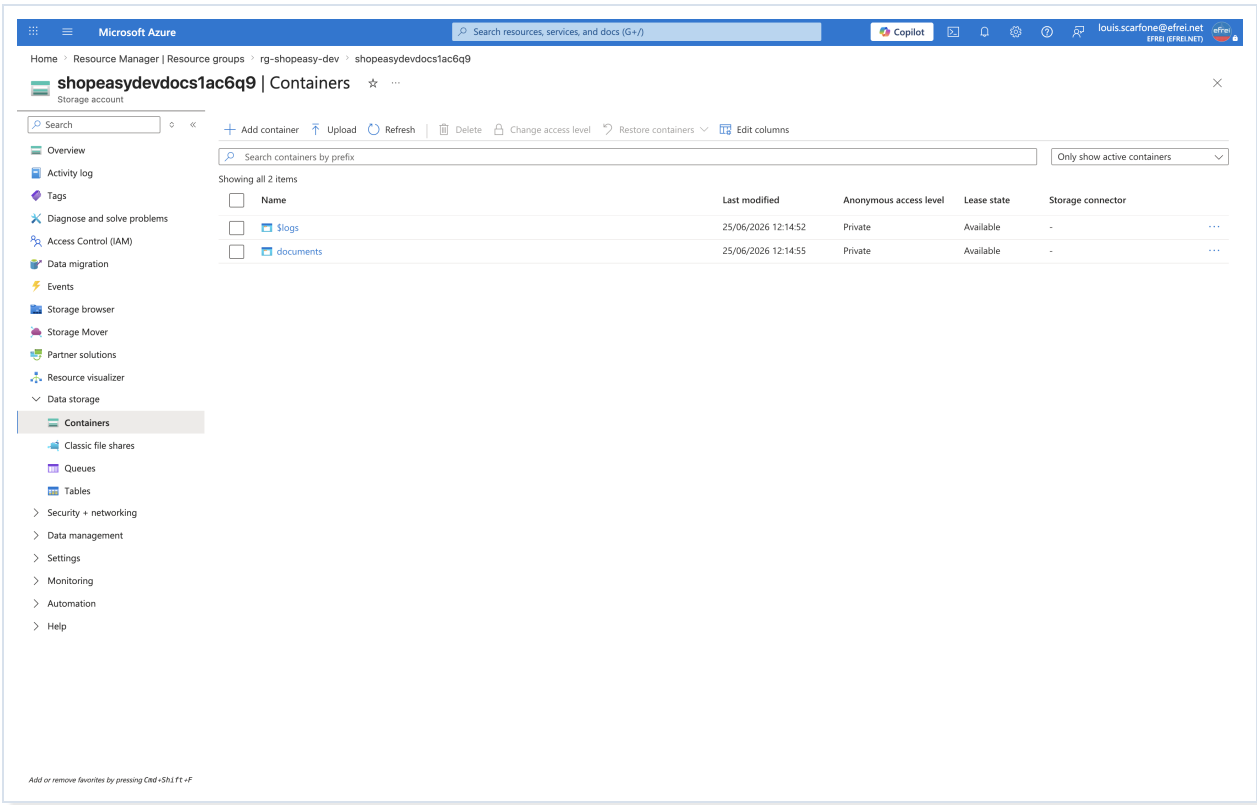
# Verifications Azure CLI
storage account : Standard_LRS, StorageV2, TLS1_2, httpsOnly=true
versioning      : isVersioningEnabled = true
container      : "documents" -> publicAccess = null (= PRIVE)
```

Le suffixe généré est `1ac6q9` → Storage Account **shopeasydevdocs1ac6q9**. Bilan : StorageV2 / Standard_LRS, TLS 1.2, HTTPS-only, **versioning activé**, et container documents **privé**.

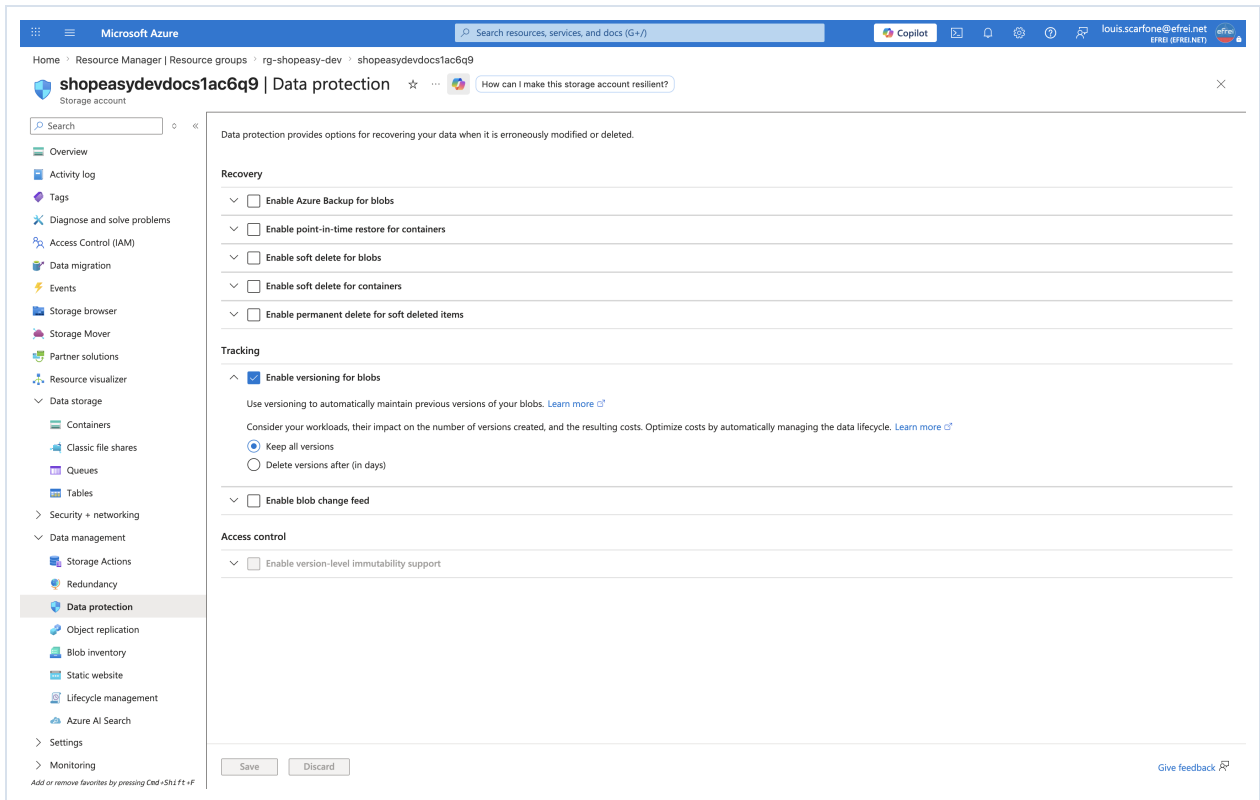
8.3 Captures portail



Storage Account shopeasydevdocs1ac6q9 (Standard_LRS, StorageV2, tags) — subscription ID flouté.



Container documents avec un niveau d'accès Private.



Versioning Blob activé (Data protection).

8.4 Questions FinOps et sécurité

1. Pourquoi le container doit-il être privé ?

Le container héberge des **documents métier sensibles** (factures, pièces clients). En accès **privé**, seuls les appels **authentifiés et autorisés** (clé de compte, jeton SAS, ou identité Azure AD via RBAC) peuvent lire les blobs. Un container **public** exposerait tous les objets à quiconque connaît l'URL → **fuite de données** et non-conformité (RGPD).

2. Pourquoi le versioning peut-il augmenter les coûts ?

À chaque modification ou suppression, le versioning **conserve une copie** de la version précédente. Ces versions **s'accumulent** et sont **facturées comme du stockage**. Sans gestion du cycle de vie, les anciennes versions ne sont **jamais nettoyées**.

3. Quelle règle de cycle de vie proposer pour limiter les coûts ?

Une **lifecycle management policy** : transférer les versions anciennes vers un niveau **Cool** puis **Archive** après N jours, et **supprimer** les versions de plus de X jours. Cela **plafonne** l'accumulation des versions et le coût associé.

8.5 Note de sécurité — accès public au niveau du compte

La vérification montre `allowBlobPublicAccess = true` **au niveau du compte** : le container est bien privé, mais le compte **autoriserait** qu'un container soit passé en public. Pour la **défense en profondeur**, on recommande de désactiver explicitement cette possibilité :

```
>_ hcl
```

```
# dans azure_rm_storage_account.docs
allow_nested_items_to_be_public = false
```

Ainsi, aucun blob ne peut être rendu public, même par erreur — mesure cohérente avec le moindre privilège (reprise dans l'analyse de sécurité de l'Atelier 13).

✓ État après l'atelier

- storage.tf créé : random_string + Storage Account shopeasydevdocs1ac6q9 (Standard_LRS, StorageV2, TLS 1.2) + container documents **privé versionné** ; apply : **3 ressources ajoutées**.
- Risque résiduel identifié : allowBlobPublicAccess = true au niveau du compte (mesure corrective proposée).
- État Terraform : **21 ressources** gérées.
- **Prêt pour l'Atelier 9 — outputs et validation finale.**

9 Atelier 9 — Outputs et validation

🎯 **Objectif** — exposer les informations utiles après déploiement et valider l'ensemble de l'infrastructure.

📦 **Livrable attendu** — outputs.tf, les sorties terraform output, la checklist technique et la page web via le Load Balancer.

9.1 Sorties exposées — outputs.tf

```
>_ hcl
```

```
output "resource_group_name" {
  description = "Nom du Resource Group cree"
  value       = azure_rm_resource_group.main.name
}
output "load_balancer_public_ip" {
  description = "IP publique du Load Balancer (point d'entree de l'application)"
  value       = azure_rm_public_ip.lb.ip_address
}
output "web_vm_public_ips" {
  description = "IP publiques des deux VM web"
  value       = azure_rm_public_ip.web[*].ip_address
}
output "storage_account_name" {
  description = "Nom du Storage Account documentaire"
  value       = azure_rm_storage_account.docs.name
}
```

Les outputs exposent les informations dont un développeur ou un script a besoin **après** le déploiement, sans fouiller le portail. L'expression `azure_rm_public_ip.web[*].ip_address` utilise le *splat* `[*]`

pour renvoyer la **liste** des IP des deux VM.

9.2 Application et lecture des sorties

Ajouter des outputs **ne modifie aucune ressource** : Terraform met seulement à jour les sorties du state.

>_ console

```
Changes to Outputs:
+ load_balancer_public_ip = "9.223.33.133"
+ resource_group_name     = "rg-shopeasy-dev"
+ storage_account_name    = "shopeasydevdocs1ac6q9"
+ web_vm_public_ips       = [ "135.225.73.252", "135.225.74.4" ]
```

```
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

```
$ terraform output
load_balancer_public_ip = "9.223.33.133"
resource_group_name     = "rg-shopeasy-dev"
storage_account_name    = "shopeasydevdocs1ac6q9"
web_vm_public_ips       = [ "135.225.73.252", "135.225.74.4" ]
```

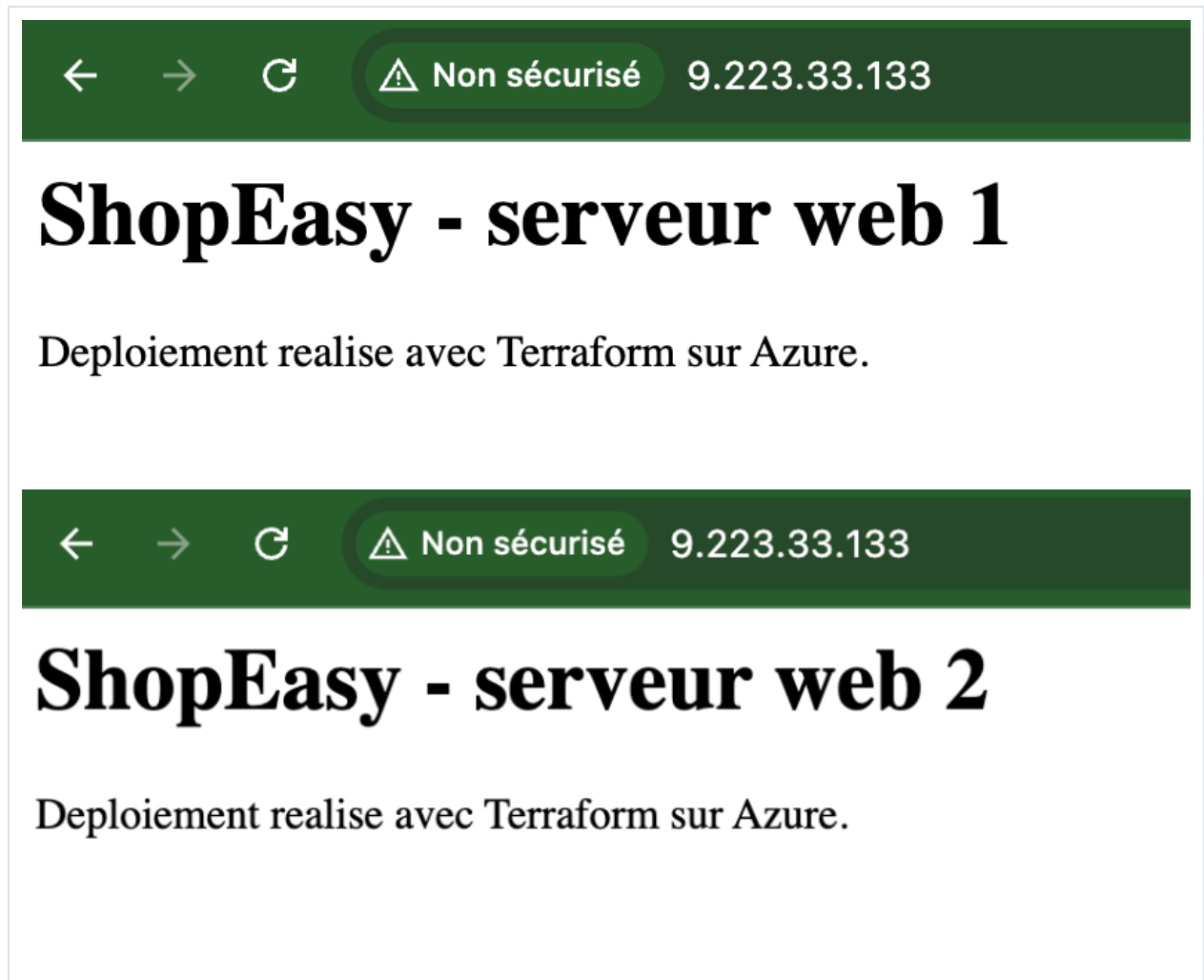
`terraform show` a également été exécuté : il restitue l'intégralité du state lisible (les 21 ressources gérées et leurs attributs). Sortie volumineuse, non reproduite ici.

9.3 Checklist technique

Contrôle	État	Preuve
Le projet Terraform s'initialise sans erreur	✓	Atelier 2 — <code>terraform init</code> : providers installés.
<code>terraform validate</code> réussit	✓	<i>Success! The configuration is valid.</i> (chaque atelier).
Le Resource Group est créé	✓	Atelier 4 — <code>rg-shopeasy-dev</code> , Succeeded.
Le VNet et le subnet existent	✓	Atelier 4 — <code>vnet-shopeasy-dev</code> + <code>snet-web</code> .
Le NSG limite SSH à l'IP admin	✓	Atelier 5 — <code>Allow-SSH-Admin 22</code> depuis <code><IP_ADMIN>/32</code> .
Deux VM Linux sont créées	✓	Atelier 6 — <code>vm-...-web-1/2</code> , VM running.
Nginx répond sur les VM	✓	Atelier 6 — <code>curl</code> → « serveur web 1 » / « 2 ».
Le Load Balancer répond en HTTP	✓	Atelier 7 — alternance 6/6 sur <code>http://9.223.33.133</code> .
Le Storage Account est privé	✓	Atelier 8 — <code>container documents publicAccess</code> : null.
Le versioning Blob est activé	✓	Atelier 8 — <code>isVersioningEnabled</code> : true.
Les ressources sont taguées	✓	<code>local.common_tags</code> (4 tags) sur RG, VNet, NSG, IP, NIC, VM, LB, Storage.

Tous les contrôles sont **au vert** (11/11).

9.4 Page web via le Load Balancer



Page web accessible via le Load Balancer (<http://9.223.33.133>) : en rafraîchissant, la réponse alterne entre les deux serveurs (répartition de charge).

✓ État après l'atelier

- `outputs.tf` créé : **4 sorties** (nom du RG, IP du LB, IP des VM, nom du Storage).
- `terraform apply` : **0 ressource modifiée**, sorties ajoutées au state; checklist technique 11/11 OK.
- Infrastructure complète : **21 ressources** gérées, déployées et validées.
- Prêt pour l'Atelier 10 — **modification de l'infrastructure**.

10 Atelier 10 — Modification de l'infrastructure

🎯 **Objectif** — modifier l'infrastructure existante (ajout d'un tag) et lire le plan pour comprendre comment Terraform applique un changement.

📦 **Livrable attendu** — modification de `locals.tf` et analyse du plan (mise à jour vs recréation).

10.1 Ajout du tag `cost_center` dans `locals.tf`

```
>_ hcl

common_tags = {
  project      = var.project
  environment  = var.environment
  owner        = "formation"
  managed_by   = "terraform"
  cost_center  = "cloud-training" # <- tag ajoute
}
```

Comme ce *local* alimente l'attribut `tags` de toutes les ressources concernées, **une seule modification** se propage à l'ensemble du projet (principe DRY).

10.2 Prévisualisation — `terraform plan`

```
>_ console

# azurerm_resource_group.main will be updated in-place
~ resource "azurerm_resource_group" "main" {
  name = "rg-shopeasy-dev"
  ~ tags = {
    + "cost_center" = "cloud-training"
      "environment" = "dev"
      "managed_by"   = "terraform"
      "owner"        = "formation"
      "project"      = "shopeasy"
  }
}
# ... (11 autres ressources taguées : VNet, NSG, IP, NIC, VM, LB, Storage)

Plan: 0 to add, 12 to change, 0 to destroy.
```

Le `~` indique une **mise à jour en place** et le `+` l'**ajout** d'une seule clé de tag ; les autres tags restent inchangés. Aucune ligne `-` (destruction) ni `-/+` (remplacement) n'apparaît.

10.3 Application et idempotence

>_ console

```
azurerms_resource_group.main: Modifications complete after 1s
...
Apply complete! Resources: 0 added, 12 changed, 0 destroyed.

# Nouveau plan (idempotence)
Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.

# Tag propage (verification)
VNet : cloud-training  VM-1 : cloud-training  LB : cloud-training  Store : cloud-training
```

10.4 Analyse du plan

1. Terraform prévoit-il de recréer toutes les ressources ?

Non. Le plan annonce 0 to add, 12 to change, 0 to destroy et chaque ressource est marquée **update in-place** (~). Un tag est un attribut **modifiable à chaud** : Terraform l'ajoute directement, sans interruption de service.

2. Quelles ressources sont simplement mises à jour ?

Les **12 ressources qui portent l'attribut tags** (= local.common_tags) : RG, VNet, NSG, les 2 IP web + l'IP du LB, les 2 NIC, les 2 VM, le LB et le Storage. Les ressources **sans tags** (subnet, associations, backend pool, sonde, règle, container, random_string) ne sont **pas concernées**.

3. Pourquoi le plan est-il indispensable avant application ?

Le plan montre **exactement** ce qui va changer **avant** d'agir : ici, des tags ajoutés en place (~), sans destruction (-) ni remplacement (-/+). Lire le plan permet de **détecter une action dangereuse** (destruction imprévue, remplacement de VM effaçant des données) avant qu'elle ne se produise. C'est le garde-fou du caractère **déclaratif** de Terraform.

✓ État après l'atelier

- locals.tf modifié : ajout du tag cost_center = "cloud-training"; apply : **12 ressources mises à jour en place**, 0 création, 0 destruction.
- Idempotence vérifiée (plan suivant : *no changes*), tag propagé à toutes les ressources taguées.
- Démonstration de la différence **mise à jour** (~) vs **recréation** (-/+).
- **Prêt pour l'Atelier 11 — observation d'une dérive (drift).**

11 Atelier 11 — Observation d'une dérive Terraform

🎯 **Objectif** — provoquer une modification manuelle hors Terraform et observer comment l'outil détecte l'écart (*drift*).

📦 **Livrable attendu** — mise en évidence de la dérive via `terraform plan` et analyse.

11.1 Création d'une dérive volontaire

Une modification est faite **manuellement dans le portail Azure**, hors Terraform : ajout du tag `manual_change = true` sur le Resource Group `rg-shopeasy-dev`. Le RG porte alors **6 tags** (les 5 déclarés + le tag manuel). Cette modification **n'existe pas dans le code** : c'est précisément ce qui constitue une dérive.

11.2 Détection — `terraform plan`

```
>_ console

# azurerm_resource_group.main will be updated in-place
~ resource "azurerm_resource_group" "main" {
  name = "rg-shopeasy-dev"
  ~ tags = {
    "cost_center" = "cloud-training"
    "environment" = "dev"
    "managed_by"  = "terraform"
    - "manual_change" = "true" -> null
    "owner"       = "formation"
    "project"     = "shopeasy"
  }
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

Terraform **compare l'état réel au code** (sa source de vérité) et détecte le tag `manual_change` ajouté hors processus. Il propose de le **supprimer** (`- "manual_change" = "true" -> null`) pour **réaligner la réalité sur l'état déclaré**.

11.3 Réconciliation — `terraform apply`

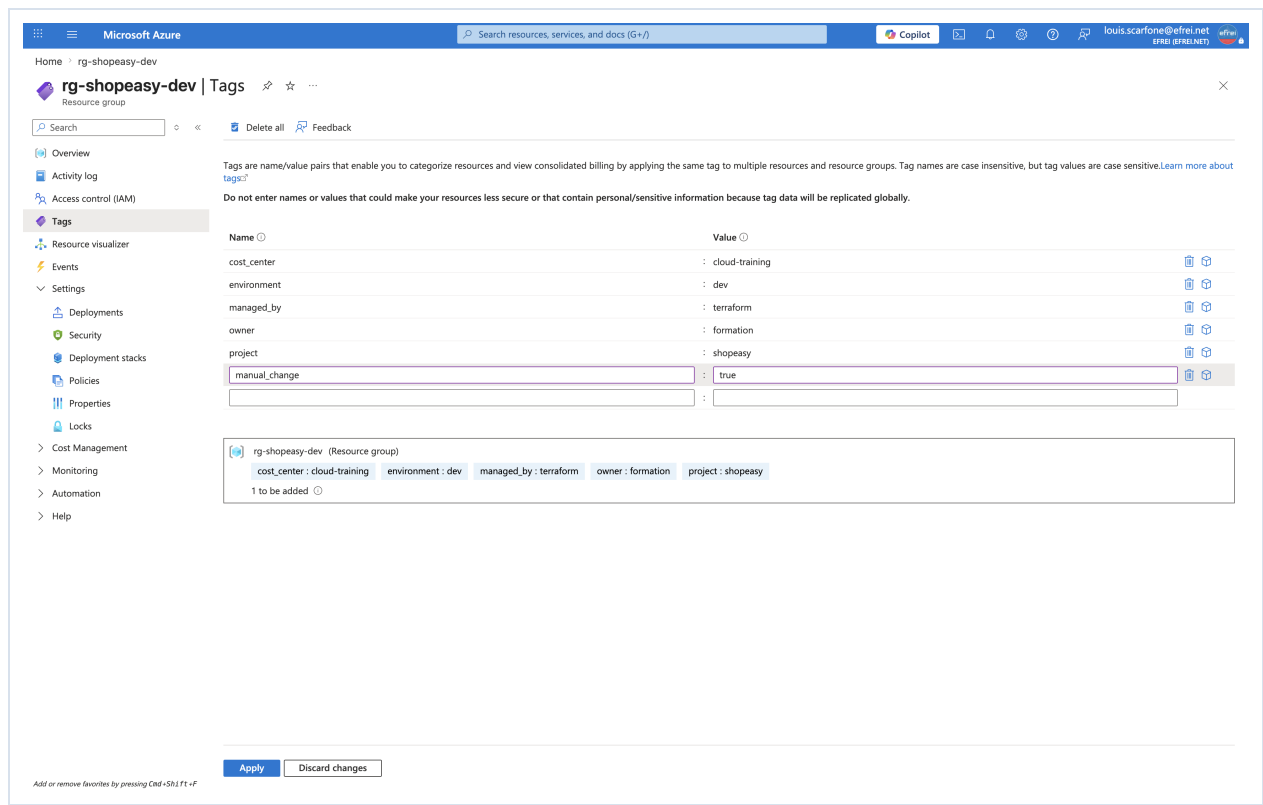
```
>_ console

azurerm_resource_group.main: Modifications complete after 1s
Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

# Tags apres reconciliation : retour aux 5 tags declares (manual_change supprime)
# Plan de controle
Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.
```

La dérive est résorbée : l'infrastructure correspond de nouveau exactement au code.

11.4 Capture portail



Tag `manual_change = true` ajouté manuellement sur le Resource Group (dérive volontaire, hors Terraform).

11.5 Analyse

1. Terraform détecte-t-il une différence ?

Oui. Lors du plan, Terraform rafraîchit l'état réel, le compare au code et **signale l'écart** : le tag `manual_change` présent dans Azure n'existe pas dans la configuration.

2. Quelle action propose-t-il ?

Une **mise à jour en place** (~) du Resource Group pour **supprimer** le tag `manual_change`, afin de revenir à l'état déclaré (0 to add, 1 to change, 0 to destroy).

3. Pourquoi les modifications manuelles sont-elles dangereuses dans une organisation ?

Elles créent une **dérive** entre le code (source de vérité) et l'infrastructure réelle : environnement **imprévisible**, **sécurité affaiblie** (une règle ouverte à la main échappe à la revue), **confiance dans le code perdue**, et un apply ultérieur peut **annuler silencieusement** un changement manuel — ou l'inverse. L'environnement devient difficile à auditer et reproduire.

4. Quelle règle d'équipe proposer pour limiter ce risque ?

Tout changement doit **passer par le code** : modification du `.tf` → **pull request** → revue → plan → apply contrôlé. On complète par : plan régulier en CI (détection des dérives), **droits portail restreints** (lecture seule en prod), **Azure Policy**, et formation des équipes. Le portail sert alors à **observer et diagnostiquer**, pas à modifier.

✓ État après l'atelier

- Dérive volontaire créée (tag `manual_change` ajouté manuellement) puis **détectée** par terraform plan (0 add, 1 change, 0 destroy).
- **Réconciliation** appliquée : retour aux 5 tags déclarés, plan de contrôle *no changes*.
- Démonstration du rôle du code comme **source de vérité** et de l'intérêt du plan.
- **Prêt pour l'Atelier 12 — préparation d'un state distant.**

12 Atelier 12 — Préparation d'un state distant

🎯 **Objectif** — comprendre pourquoi et comment externaliser le state Terraform dans un backend distant sécurisé.

📦 **Livrable attendu** — structure cible du backend Azure et analyse (atelier de **préparation conceptuelle**, sans migration du state).

12.1 Le state local et ses limites

Le projet utilise un **state local** : un fichier `terraform.tfstate` sur le poste. Ce state **contient des données sensibles en clair** — une recherche des **noms** d'attributs sensibles (sans afficher aucune valeur) le confirme :

>_ console

```
$ ls -la terraform.tfstate          # 59 639 octets
$ git check-ignore terraform.tfstate # -> ignore (non versionne)
$ terraform state list | wc -l      # 21 ressources suivies

# Recherche de noms d'attributs sensibles dans le state :
cle presente: "admin_password"
cle presente: "primary_access_key"    <- cle d'accès du Storage Account
cle presente: "secret"
```

Le state local est acceptable en découverte individuelle, mais **insuffisant en équipe** :

- **perte** possible du fichier ; **conflits** entre plusieurs administrateurs ;
- **pas de verrouillage** → corruption en cas d'apply simultané ;
- **présence de secrets** → un commit accidentel exposerait clés et mots de passe ;
- **difficile à intégrer** dans une chaîne CI/CD.

12.2 Structure cible — backend Azure Storage

```
>_ hcl

terraform {
  backend "azurerm" {
    resource_group_name = "rg-terraform-state"
    storage_account_name = "stterraformstatexxx"
    container_name      = "tfstate"
    key                 = "shopeasy/dev/terraform.tfstate"
  }
}
```

Champ	Rôle
resource_group_name	RG dédié au state (séparé des ressources applicatives).
storage_account_name	Storage Account hébergeant le state (nom globalement unique).
container_name	Container Blob tfstate.
key	Chemin/nom du blob de state — inclut l'environnement (shopeasy/dev/...) pour isoler dev/recette/prod.

Le backend azurerm fournit nativement le **verrouillage** (via un *lease* sur le blob) : deux apply simultanés ne peuvent pas corrompre le state.

⚠ Bootstrap — ne pas ajouter le bloc à la légère

Le bloc backend ne doit **pas** être ajouté sans avoir créé **au préalable** le Resource Group, le Storage Account et le container de state. Dans un vrai projet, cette initialisation est réalisée par une **équipe plateforme** ou un **bootstrap dédié**, puis sécurisée (RBAC, chiffrement, réseau). La bascule se ferait ensuite avec `terraform init -migrate-state`. **Cette migration n'est pas effectuée ici** : l'atelier reste conceptuel, et le state du TP demeure local (puis supprimé à l'Atelier 14).

12.3 Questions

1. Pourquoi le state distant facilite-t-il le travail en équipe ?

Il fournit un **emplacement central et partagé** : tous les membres et la CI/CD lisent et écrivent le **même** state, sans copies divergentes. Il apporte le **verrouillage** (un seul apply à la fois), supprime le state des postes (**pas de perte ni de commit accidentel**) et s'intègre dans un **pipeline**.

2. Pourquoi faut-il protéger l'accès au Storage Account du state ?

Parce que le state **contient des secrets en clair** (admin_password, primary_access_key, secret) et l'inventaire complet de l'infrastructure. Il faut : **RBAC restreint, chiffrement au repos, restrictions réseau** (private endpoint / pare-feu), désactivation de l'accès public, rotation des clés et journalisation.

3. Pourquoi faut-il séparer le state de développement, recette et production ?

Pour **isoler les environnements** : une erreur sur le state **dev** ne doit jamais impacter la **prod**.

La séparation (clé/container distincts, ex. shopeasy/prod/terraform.tfstate) permet des **droits différenciés**, évite d'**appliquer par erreur** un changement dev sur la prod, et donne à chaque environnement un **cycle de vie indépendant**.

✓ État après l'atelier

- Limites du state local démontrées : fichier local de 21 ressources **contenant des secrets en clair**.
- Structure cible documentée : backend azurearm (Storage Account + container tfstate, clé par environnement, verrouillage natif) ; bootstrap et migration expliqués (**non appliqués**).
- **Prêt pour l'Atelier 13 — analyses coût, sécurité et maintenabilité.**

13 Atelier 13 — Analyse coût, sécurité et maintenabilité

🎯 **Objectif** — analyser l'infrastructure déployée sous trois angles : coûts (FinOps), sécurité et maintenabilité.

📦 **Livrable attendu** — tableau FinOps, tableau d'analyse de sécurité et réponses sur la maintenabilité.

13.1 Analyse FinOps

Prix unitaires **réels** (Azure Retail Prices API, région swedencentral, pay-as-you-go USD, hypothèse 24/7 = 730 h/mois) :

Poste	Prix unitaire officiel
VM Standard_B2ats_v2 (Linux)	0,00972 \$/h
IP publique Standard (static)	0,005 \$/h
Disque OS Standard HDD S4 (32 Go) LRS	1,536 \$/mois
Load Balancer Standard	≈ 0,025 \$/h (règles incluses)
Blob Storage (faible volume)	négligeable

Ressource	Coût relatif	Risque de surcoût	Optimisation proposée
VM Linux (2 × B2ats_v2)	Moyen-élevé (≈ 14,2 \$)	VM allumées 24/7 sans usage réel	Désallouer hors période ; autoscale en prod ; taille adaptée
IP publiques (3)	Moyen (≈ 10,9 \$)	Facturées même à trafic nul	Réduire à 1 (celle du LB) ; VM sans IP publique + Bastion
Load Balancer (Standard)	Élevé (≈ 18,2 \$)	Facturé en continu, même sans trafic	Garder pour la prod ; App Gateway si L7/WAF requis
Storage Account (LRS)	Faible (< 0,1 \$)	Réplication géo (GRS) inutile en dev	LRS (pas GRS) ; suppression au nettoyage
Versioning Blob	Faible → croissant	Accumulation illimitée d'anciennes versions	Lifecycle : transition Cool/Archive + purge des versions
Disques managés (2 × OS, S4)	Faible (≈ 3,1 \$)	Disques orphelins après suppression VM	Taille adaptée ; suppression au nettoyage

⚠ Total et postes principaux

Total indicatif ≈ 46,5 \$/mois en 24/7. Les 3 postes principaux sont le **Load Balancer (18,2 \$)**, le **compute des 2 VM (14,2 \$)** et les **IP publiques (10,9 \$)** — le LB et les IP facturent même à trafic nul. Désallouer les VM annule leur coût compute, et la **suppression du Resource Group** (Atelier 14) ramène le coût à ≈ 0.

13.2 Analyse de sécurité

Risque	Cause possible	Impact	Correction
SSH trop ouvert	Source 0.0.0.0/0 au lieu de l'IP admin	Scans, brute-force, compromission	Restreindre à l'IP /32 (<i>appliqué</i>) ; Bastion / JIT en prod
State exposé	tfstate commité ou Storage de state non protégé	Fuite de secrets + inventaire du SI	.gitignore (<i>appliqué</i>) ; backend distant chiffré + RBAC
Storage public	Container public ou allowBlobPublicAccess=true	Fuite de documents métier (RGPD)	Container privé (<i>appliqué</i>) ; allow_nested_items_to_be_public=false
Tags absents	Oubli de tags sur des ressources	Coûts non imputables, gouvernance impossible	common_tags systématiques (<i>appliqué</i>) ; Azure Policy
Secrets dans Git	Mot de passe/clé en dur dans .tf ou tfvars versionné	Compromission d'identifiants	Aucun secret en dur ; tfvars ignoré (<i>appliqué</i>) ; Key Vault

Les corrections (*appliqué*) sont déjà en place (Ateliers 1, 5, 8, 11). Les autres sont les évolutions recommandées pour la production.

13.3 Maintenabilité

1. Comment rendre ce projet réutilisable pour un environnement de recette ?

Le code est déjà **paramétré par variables** : il suffit de fournir un **terraform.tfvars** par environnement (ex. `environment = "recette"`, sa région, sa taille de VM) et un **state séparé** (clé/backend distincts).

Le **code** reste **unique**; seules les **valeurs** changent. On peut aussi utiliser des **workspaces**.

2. Quelles variables faudrait-il ajouter?

`vm_size`, `vm_count` (nombre d'instances), `address_space / web_subnet_prefix`, `storage_replication_type`, `enable_public_ip` (booléen pour supprimer les IP publiques en prod), et un bloc de **tags** **additionnels**.

3. Quels fichiers pourraient devenir des modules Terraform?

`network.tf + security.tf` → **module network**; `compute.tf` → **module compute**; `loadbalancer.tf` → **module loadbalancer**; `storage.tf` → **module storage**. Appelés depuis un `main.tf` avec des variables, ils deviennent **réutilisables** pour dev / recette / prod.

4. Quelles validations automatiques pourrait-on ajouter dans une pipeline CI/CD?

`terraform fmt -check` et `validate`; **tflint**; **tfsec** / **checkov** (sécurité statique); **gitleaks** (secrets); `terraform plan` automatique sur chaque **pull request** + revue et approbation avant `apply`; exécution régulière de `plan` pour détecter les dérives.

✓ État après l'atelier

- **FinOps** : coût ≈ **46,5 \$/mois** (24/7, prix réels), 3 postes principaux (LB, compute, IP), optimisations par ressource.
- **Sécurité** : 6 risques analysés (cause / impact / correction), dont 4 déjà corrigés dans le projet.
- **Maintenabilité** : réutilisation par environnement, variables à ajouter, modularisation, validations CI/CD.
- **Prêt pour l'Atelier 14 — nettoyage de l'environnement.**

14 Atelier 14 — Nettoyage de l'environnement

- 🎯 **Objectif** — supprimer proprement toutes les ressources créées pour arrêter la facturation.
- 📦 **Livrable attendu** — prévisualisation de la destruction et commande `terraform destroy`.

✓ Destruction exécutée

Après validation de l'ensemble des livrables et des captures, la destruction effective a été lancée (`terraform destroy`). Les **21 ressources ont été supprimées** et le Resource Group n'existe plus côté Azure — la **facturation est arrêtée** (détaillée ci-dessous).

14.1 Pourquoi détruire l'environnement?

L'infrastructure de test coûte ≈ **46,5 \$/mois en 24/7** (cf. Atelier 13), facturée **même à trafic nul** pour le Load Balancer et les IP publiques. Comme tout est décrit en **Infrastructure as Code**, l'environnement est **reproductible à l'identique** par `terraform apply` : il n'y a aucune raison de le laisser tourner entre deux séances. La destruction ramène le coût à ≈ 0.

14.2 Prévisualisation — terraform plan -destroy

>_ console

```
$ terraform plan -destroy          # ne supprime rien : liste ce qui SERAIT détruit
azurerm_lb.web, azurerm_lb_backend_address_pool.web, azurerm_lb_probe.http,
azurerm_lb_rule.http, azurerm_linux_virtual_machine.web[0..1],
azurerm_network_interface.web[0..1],
azurerm_network_interface_backend_address_pool_association.web[0..1],
azurerm_network_security_group.web, azurerm_public_ip.lb,
azurerm_public_ip.web[0..1], azurerm_resource_group.main,
azurerm_storage_account.docs, azurerm_storage_container.documents,
azurerm_subnet.web, azurerm_subnet_network_security_group_association.web,
azurerm_virtual_network.main, random_string.suffix          ... will be destroyed

Plan: 0 to add, 0 to change, 21 to destroy.
```

Les **21 ressources** gérées par Terraform sont prévues pour suppression.

14.3 Exécution et vérification

Après validation de tous les livrables et captures, la destruction a été exécutée (terraform destroy -auto-approve) : elle supprime toutes les ressources gérées par le state, dans l'ordre inverse des dépendances. Sortie réelle (extrait, identifiant d'abonnement masqué) :

>_ console

```
random_string.suffix: Destruction complete after 0s
azurerm_lb_rule.http: Destruction complete after 4s
azurerm_network_security_group.web: Destruction complete after 11s
azurerm_lb.web: Destruction complete after 10s
azurerm_linux_virtual_machine.web[0]: Destruction complete after 32s
azurerm_linux_virtual_machine.web[1]: Destruction complete after 32s
azurerm_public_ip.lb / web[0..1]: Destruction complete after ~11s
azurerm_network_interface.web[0..1]: Destruction complete after ~11-22s
azurerm_subnet.web: Destruction complete after 11s
azurerm_virtual_network.main: Destruction complete after 11s
azurerm_resource_group.main: Destruction complete after 21s

Destroy complete! Resources: 21 destroyed.
```

Vérifications après destruction :

>_ console

```
$ terraform state list
(vide -- aucune ressource gérée)

$ az group show -n rg-shopeasy-dev
ERROR: (ResourceGroupNotFound) Resource group 'rg-shopeasy-dev' could not be found.
```

⚠ Point obligatoire du TP

Ne jamais terminer une séance sans avoir supprimé les ressources de formation. C'est fait : les **21 ressources sont détruites**, le Resource Group n'existe plus, la **facturation est arrêtée**.

✓ État après l'atelier

- terraform destroy exécuté : **Destroy complete! Resources : 21 destroyed.**
- État Terraform **vide**; Resource Group rg-shopeasy-dev **absent** de l'abonnement (ResourceGroupNotFound).
- **Facturation arrêtée** (≈ 0); l'environnement reste **reproductible à l'identique** par terraform apply.
- **Fin du TP2** : projet Terraform complet — déployé, validé, documenté et nettoyé.

15 Note technique — synthèse des choix

Objet — industrialisation du déploiement Azure de ShopEasy avec Terraform (Infrastructure as Code)

Portée — synthèse des choix techniques, de sécurité, de coût et de gouvernance, et proposition d'évolution

Auteurs (binôme) — Louis SCARFONE & Maxence BOURRAGUÉ

15.1 Contexte

Le TP1 avait conçu et déployé **manuellement** (Azure CLI) l'architecture cible de ShopEasy. Le TP2 reprend cette architecture et la décrit en **Infrastructure as Code** : l'infrastructure devient un ensemble de fichiers **.tf versionnés, relus, appliqués puis détruits** de manière reproductible. Le résultat n'est pas seulement « une infrastructure qui fonctionne », mais une infrastructure **décrite proprement, lisible, versionnable, sécurisée et maîtrisée dans son cycle de vie**.

15.2 Architecture déployée

Couche	Ressources
Organisation	Resource Group <code>rg-shopeasy-dev</code> (<code>swedencentral</code>), tags de gouvernance.
Réseau	VNet <code>10.20.0.0/16</code> , subnet <code>snet-web 10.20.1.0/24</code> , NSG (HTTP ouvert, SSH restreint à l'IP admin).
Calcul	2 VM Linux Standard_B2ats_v2 (Ubuntu 22.04), Nginx installé par cloud-init, déployées via count.
Répartition	Load Balancer Standard (IP publique, backend pool, sonde HTTP, règle 80).
Stockage	Storage Account StorageV2 LRS , container privé , versioning Blob activé.
Exploitation	Outputs (IP du LB, IP des VM, nom du Storage), state local protégé.

21 ressources Terraform, déployées et validées (checklist technique de l'Atelier 9 : 11/11).

15.3 Choix techniques

Choix	Décision	Justification
Outil IaC	Terraform + <code>azurerem</code> ~> 4.0	Déclaratif, multi-cloud, lisible.
Authentification	<code>az login</code> + <code>ARM_SUBSCRIPTION_ID</code>	Aucun identifiant en dur dans le code.
Région	swedencentral	<code>francecentral</code> bloquée (policy Azure for Students).
Taille de VM	Standard_B2ats_v2	B1s indisponible ; B2ats_v2 recommandée Microsoft.
Multiplication	<code>count = 2</code> pour IP/NIC/VM	Deux serveurs identiques sans duplication (DRY).
Provisionnement	<code>cloud-init</code> + <code>templatefile</code> + <code>custom_data</code>	Installation Nginx automatique, page par serveur.
Clé SSH	<code>file(pathexpand(...))</code>	<code>file()</code> ne développe pas ~.
Nommage / tags	<code>local.prefix</code> + <code>local.common_tags</code>	Source unique, cohérence et gouvernance.
Lisibilité	Découpage par responsabilité (<code>network</code> , <code>compute</code> ...)	Projet relisible et maintenable.

15.4 Sécurité

SSH **restreint** à l'IP admin (/32); **aucun secret dans le code** (terraform.tfvars exclu du dépôt, .example anonymisé versionné); **state protégé** (gitignore; il contient admin_password, primary_access_key, secret en clair); **Storage privé**, TLS 1.2, HTTPS-only; captures **anonymisées** (IP admin et identifiants Azure floutés). **Recommandations production** : Azure Bastion, backend de state distant chiffré + RBAC, allow_nested_items_to_be_public = false, Application Gateway + WAF en HTTPS, Azure Policy d'obligation de tags.

15.5 FinOps

Coût $\approx$ 46,5 \$/mois en 24/7 (prix réels Azure Retail Prices). Postes principaux : **Load Balancer** ($\approx$ 18 \$), **compute des 2 VM** ($\approx$ 14 \$), **IP publiques** ($\approx$ 11 \$) — le LB et les IP facturent même à trafic nul. Leviers : désallouer les VM hors usage, réduire les IP publiques (Bastion), lifecycle policy sur le Blob, et terraform destroy entre les séances pour ramener le coût à $\approx$ 0.

15.6 Gouvernance et cycle de vie

Workflow : init $\rightarrow$ fmt $\rightarrow$ validate $\rightarrow$ plan $\rightarrow$ apply $\rightarrow$ destroy, le plan étant systématiquement relu. La modification d'un tag se propage en **mise à jour en place** (~), sans recreation. Une modification manuelle (portail) est **détectée** par terraform plan et **réconciliée**, démontrant le rôle du code comme **source de vérité**.

15.7 Amélioration proposée (mise en autonomie — Option B : Azure Bastion)

Objectif : supprimer les IP publiques directes des VM et administrer via **Azure Bastion**, pour réduire la surface d'attaque et le coût des IP.

- **Modification** : ajout d'un subnet AzureBastionSubnet (10.20.255.0/27) + un azurerm_bastion_host (+ IP publique Standard); **suppression des IP publiques des VM**; retrait de la règle SSH publique du NSG.
- **Fichiers impactés** : compute.tf (NIC sans IP publique), network.tf (subnet Bastion), security.tf (retrait Allow-SSH-Admin), nouveau bastion.tf, variables.tf (enable_public_ip).
- **Risques** : le Bastion a un **coût horaire** (à arbitrer en dev); le test web direct par VM n'est plus possible (uniquement via le LB); la bascule nécessite un plan attentif (mise à jour des NIC, pas recreation des VM).

Conclusion

Le TP2 fait passer ShopEasy d'une infrastructure **créée à la main** à une infrastructure **décrite en code**, reproductible et gouvernable. Les principes clés — **moindre privilège, pas de secret versionné, nommage et tags cohérents, lecture systématique du plan, maîtrise du drift et des coûts** — sont appliqués et documentés. Les évolutions proposées (Bastion, state distant, modularisation, CI/CD) rapprocheraient l'environnement d'un usage de production.

16 Quiz de validation

1. **Terraform est-il un outil impératif ou déclaratif ? Justifier.**
Déclaratif : on décrit l'état final souhaité, Terraform calcule les actions (créer/modifier/détruire) pour l'atteindre — d'où l'importance du plan.
2. **Quel est le rôle d'un provider Terraform ?**
Un plugin qui dialogue avec l'API d'une plateforme et traduit les ressources Terraform en appels concrets (ici `azurerm` pour Azure).
3. **À quoi sert le fichier `terraform.tfstate` ?**
C'est le fichier d'état : il fait le lien entre le code et les ressources réelles créées dans Azure.
4. **Pourquoi exécuter `terraform plan` avant `terraform apply` ?**
Pour prévisualiser les changements avant d'impacter Azure ; une destruction ou un remplacement imprévu est un signal d'alerte.
5. **Quelle commande formate le code Terraform ?**
`terraform fmt`.
6. **Quelle commande vérifie la syntaxe et la cohérence de base ?**
`terraform validate`.
7. **Quel service Azure correspond au réseau privé logique ?**
Le Virtual Network (VNet).
8. **Quel composant Azure filtre les flux réseau entrants et sortants ?**
Le Network Security Group (NSG).
9. **Pourquoi restreindre SSH à une seule adresse IP ?**
Pour réduire la surface d'attaque (scans et force brute) : seule l'IP admin peut tenter une connexion.
10. **Quel est l'intérêt de `count` dans le déploiement des VM ?**
Déployer plusieurs instances identiques sans dupliquer le code ; `count.index` nomme et relie chaque VM à son interface et son IP.
11. **Pourquoi utiliser des variables ?**
Pour éviter les valeurs codées en dur et rendre le projet paramétrable et réutilisable sur plusieurs environnements.
12. **Pourquoi utiliser des outputs ?**
Pour exposer les informations utiles après déploiement (IP du LB, IP des VM, nom du Storage) sans fouiller le portail.
13. **Pourquoi taguer les ressources ?**
Pour la gouvernance et le FinOps : imputer les coûts, filtrer les ressources, identifier le propriétaire et l'environnement.
14. **Différence entre un changement Terraform et un changement manuel dans le portail ?**
Le changement Terraform est versionné, relu et tracé (`plan/apply`) ; le changement manuel se fait hors code → dérive non tracée.
15. **Quel risque présente un Storage Account public ?**
Une fuite de données : les blobs deviennent accessibles à quiconque connaît l'URL (risque RGPD).
16. **Pourquoi le versioning Blob peut-il générer des coûts supplémentaires ?**
Il conserve une copie de chaque version à chaque modification ; ces versions s'accumulent et sont facturées comme du stockage.
17. **À quoi sert un Load Balancer ?**
À répartir le trafic entrant sur plusieurs VM derrière un point d'entrée unique, ce qui améliore la disponibilité et permet de monter en charge.

18. **Pourquoi un state distant est-il préférable en équipe ?**
Emplacement central partagé, verrouillage (pas de corruption), pas de state sur les postes, intégration CI/CD.
19. **Citer deux bonnes pratiques de sécurité pour un projet Terraform.**
Ne jamais versionner de secret (tfvars/tfstate exclus, Key Vault) ; moindre privilège (SSH restreint, NSG, backend de state distant sécurisé).
20. **Citer deux améliorations pour rapprocher l'architecture d'un environnement de production.**
Supprimer les IP publiques des VM + Azure Bastion et externaliser le state ; Application Gateway + WAF (HTTPS), supervision (Azure Monitor) et pipeline CI/CD.

🚩 **Fin du dossier** — les 14 ateliers, la note technique et le quiz de validation sont complets : projet Terraform initialisé et paramétré, déploiement Azure (réseau, NSG, 2 VM cloud-init, Load Balancer, Storage privé versionné), outputs, modification et dérive maîtrisées, state distant, analyses coût/sécurité/maintenabilité et nettoyage.